Entire System Design & Development Plan

SwiftDrop

Parcel Delivery & Courier Management System

Version 3.0 | 31 October 2026 | Proposed implementation baseline

Decision	Baseline
Team	Nine members; leader owns M1 Identity and Access.
Architecture	Layered architecture in Java: one Spring Boot backend, one React frontend, one PostgreSQL database and RabbitMQ.
Data and integration	PostgreSQL, RabbitMQ, REST, SSE, Google OIDC and merchant API keys.
Status	Design only. This document does not claim that the application, deployment or tests are complete.

Purpose and authority

This beginner-oriented document covers the entire system, not only the leader's work. It translates the supplied assignment into an implementable design. The assignment takes precedence over the later enterprise SRS where they differ. Design choices below are proposals, not additional marking requirements.

The original brief specifies ten members; the user specifies nine. Obtain lecturer confirmation for the team-size difference. Names, deadline, hosting target and actual business rates remain to be supplied.

Scope corrections from the enterprise proposal

Keep password signup/login, Google login and application JWTs: all are required by the brief. Use the assignment lifecycle CREATED, PICKED_UP, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERED and FAILED. COD settlements, returns, bulk imports, route optimization, offline courier synchronization and international shipping are later extensions, not baseline commitments.

Reading map

Read first: page 4 (layers), page 27 (worked example), page 28 (folders) and page 29 (starting plan). Full design: pages 2-16. Every member: M1 page 17; M2-M9 pages 19-26. Verification: page 18. Risks and documentation plan: pages 30-31. Marking checklist: page 32; full architecture, detailed ER, patterns and notification sequence: pages 33-38.

Scope and requirement traceability

Assignment requirement	Design / owner	Acceptance evidence
Signup, password login, Google, JWT and RBAC	M1; sections 10, 14, 17	Real Google demo; invalid/expired token and role-denial tests.
Quote, booking, history and receipt	M3 + M4; sections 7, 9, 11	Quote confirmed once; own history; delivered receipt.
Merchant approval and secured order API	M2; sections 9, 12	Pending merchant denied; approved key creates one order.
Assign deliveries and monitor workload	M5; sections 6, 8, 13	One active assignment; correct active-job counts.
Courier status, proof and failure reason	M6; sections 8, 13	Only assigned courier updates a legal transition.
Public mobile tracking and live updates	M7; sections 9, 15, 16	SSE updates an open mobile-sized tracking page.
Async notification and merchant webhook	M8 + M9; sections 13, 15	Real RabbitMQ event reaches both consumers; retry demonstrated.
Use cases, ER, layers and four flow diagrams	Sections 4-8 and 11-14	Diagrams correspond to final implementation.
Docker, tests, coverage and contribution sheet	All members; section 18	Clean Compose startup and actual reports.

Baseline business decisions

FAILED is terminal in the first release. A delivered parcel requires recipient name or approved proof; photo upload is optional. The downloadable delivery receipt is available after DELIVERED and is not a payment receipt. Demo prices use LKR minor units and integer grams. Quotes expire after ten minutes; rates are synthetic until agreed.

Later scope

The competitor audit motivates clear states, recoverable forms and fast lists. It does not establish that competitors have specific defects. Bulk uploads and COD ledgers are useful future capabilities, but expanding into them requires revised ownership, data models and acceptance tests.

Team ownership within the layered application

Owner	Full-stack feature	Owned table group
M1 - leader	Signup, login, Google, profile, user administration	identity
M2	Merchant onboarding, approval, API keys and order dashboard	merchant
M3	Zones, rate rules, quote UI and calculation	pricing
M4	Booking, history and delivery receipts	booking
M5	Courier work profiles, dispatch, workloads and reports	dispatch
M6	Courier tasks, canonical lifecycle, history and proof	execution
M7	Public and authenticated tracking; SSE projection	tracking
M8	Notification templates, consumer and delivery logs	notification
M9	Webhook settings, signed delivery, retries and attempt logs	webhook

One architecture; nine feature responsibilities

Layered architecture describes how the code is organized: controllers, business services and repositories. A monolith describes deployment: one backend application. These ideas are compatible. This revision uses a simple layered Java monolith; the nine features are work allocations, not nine services or nine applications.

Each teammate works across all layers for their assigned feature. The feature owner maintains its controllers, services, repositories, tables, React screens and tests. M5 owns courier work profiles; M1 owns login identity; M2 owns merchant approval. Shared business operations call service methods rather than issuing HTTP requests to the same backend.

Database and collaboration simplification

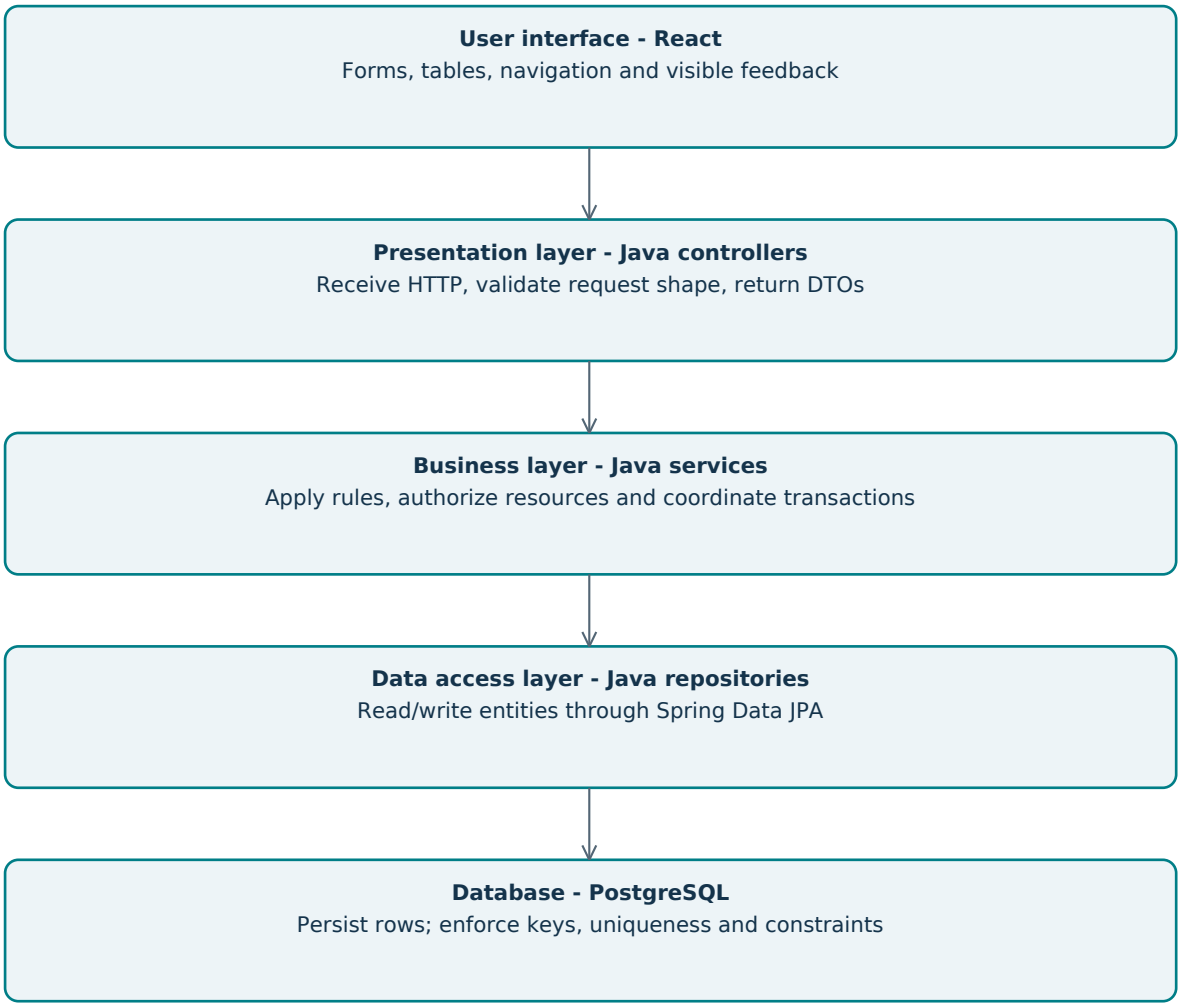
Use one PostgreSQL database and one application schema for the first release, with table ownership documented. Enforce foreign keys between related tables, even when different members own them. Coordinate migrations centrally. This replaces the previous nine-schema proposal. Controllers never use repositories directly; services use their feature repositories and other published services.

Independent testing without nine runtimes

Each member runs the same application against a private local database, plus focused tests with mocked dependency services. Use deterministic development fixtures for unfinished screens. Entirely independent applications are not required. A mock is a temporary replacement that returns agreed example data; it must never become production authentication or pretend that an unfinished integration is complete.

No one needs to finish their whole feature before another starts. Freeze identifiers, DTOs and service contracts early, develop using mocks, and integrate one small working flow at a time. The leader coordinates integration; all members share review and setup work.

Layered architecture with Java - explained



Request direction: React -> Controller -> Service -> Repository -> Database. Results return upward as response DTOs and JSON. Layers are code responsibilities, not separate servers. The database runs separately; controller/service/repository classes run inside the same Spring Boot application.

Supporting components

Spring Security filters authenticate before controllers. DTOs describe data exchanged with clients; entities describe stored records. Exception handling produces safe consistent errors. Integration adapters call Google, RabbitMQ and notification/webhook providers. Consumers call business services, never controllers.

Minimal deployment

Start with four Compose containers: React/static-web reverse proxy, Java backend, PostgreSQL and RabbitMQ. Run bounded asynchronous consumers in the backend initially. A separate worker process is optional later; it is not a prerequisite for the assignment. Persist uploads privately only if photo proof is implemented. Expose the web entry point; keep database and broker private.

Implementation choices

Java + Spring Boot, Maven, Spring Web, Spring Security, Spring Data JPA, Bean Validation and Flyway are proposed implementation tools. React remains the browser UI; Java is the backend language. Pin one compatible JDK/Spring Boot combination for the team; do not select each Spring dependency version independently. Repository and transaction concepts are described in [T1-T3], page 31.

Use-case diagram: customer and merchant

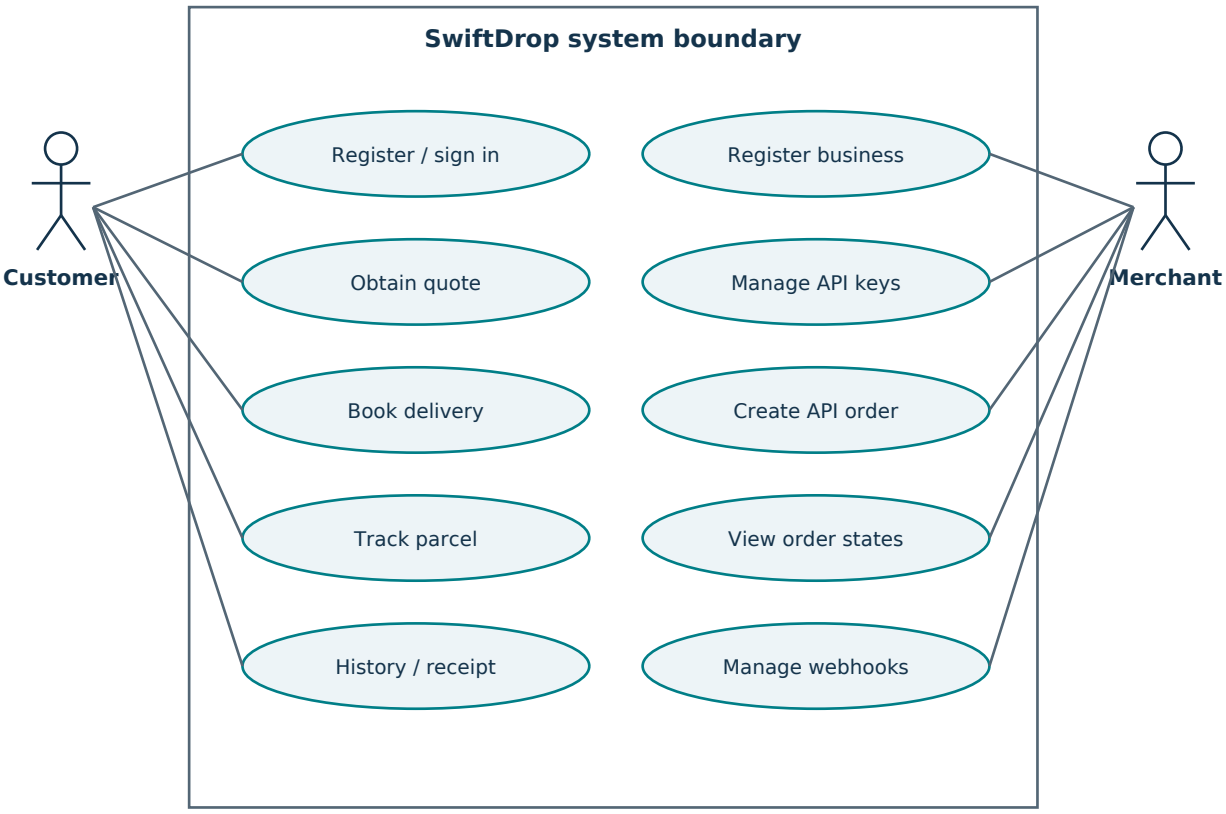
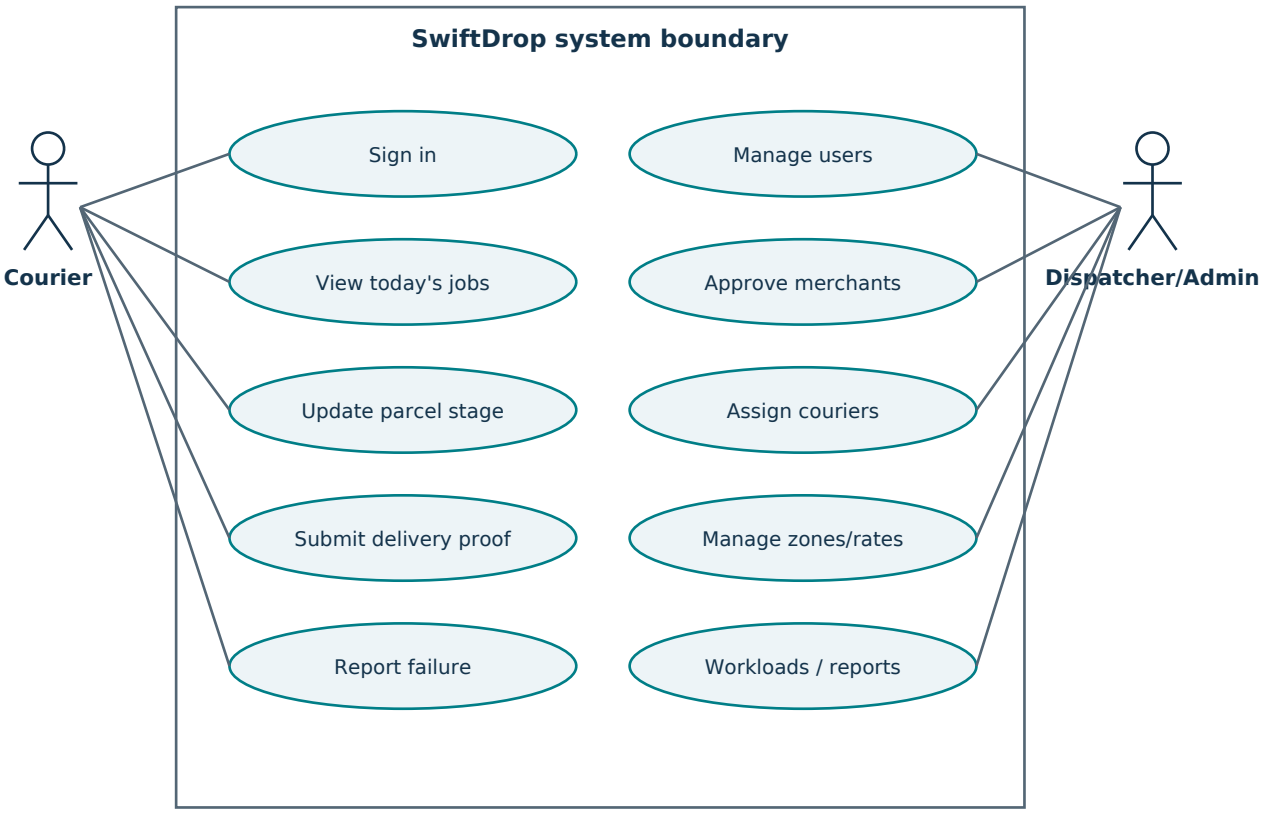


Diagram notation: stick figures are actors; ellipses are use cases; solid associations indicate participation. Authentication and approval are preconditions where applicable, not repeated login steps in every use case. A merchant API client acts on behalf of the approved merchant.

Use case	Precondition and result
Book delivery	Authenticated customer; supported route; valid owned quote. Creates one parcel, lifecycle and tracking reference.
Create API order	Active key for an approved merchant. The same booking workflow applies; external reference and idempotency prevent duplicates.
Track parcel	Anonymous viewer may use a high-entropy public number. Result excludes private addresses, contacts and proof.
Manage webhooks	Approved merchant manages only its subscriptions, secret rotation and delivery logs.

External actors: Google authenticates social-login users; merchant receivers accept signed callbacks; the notification provider receives outbound messages. Their interactions are detailed in sections 14-15. Public tracking also serves recipients who do not hold an account.

Use-case diagram: courier and dispatcher

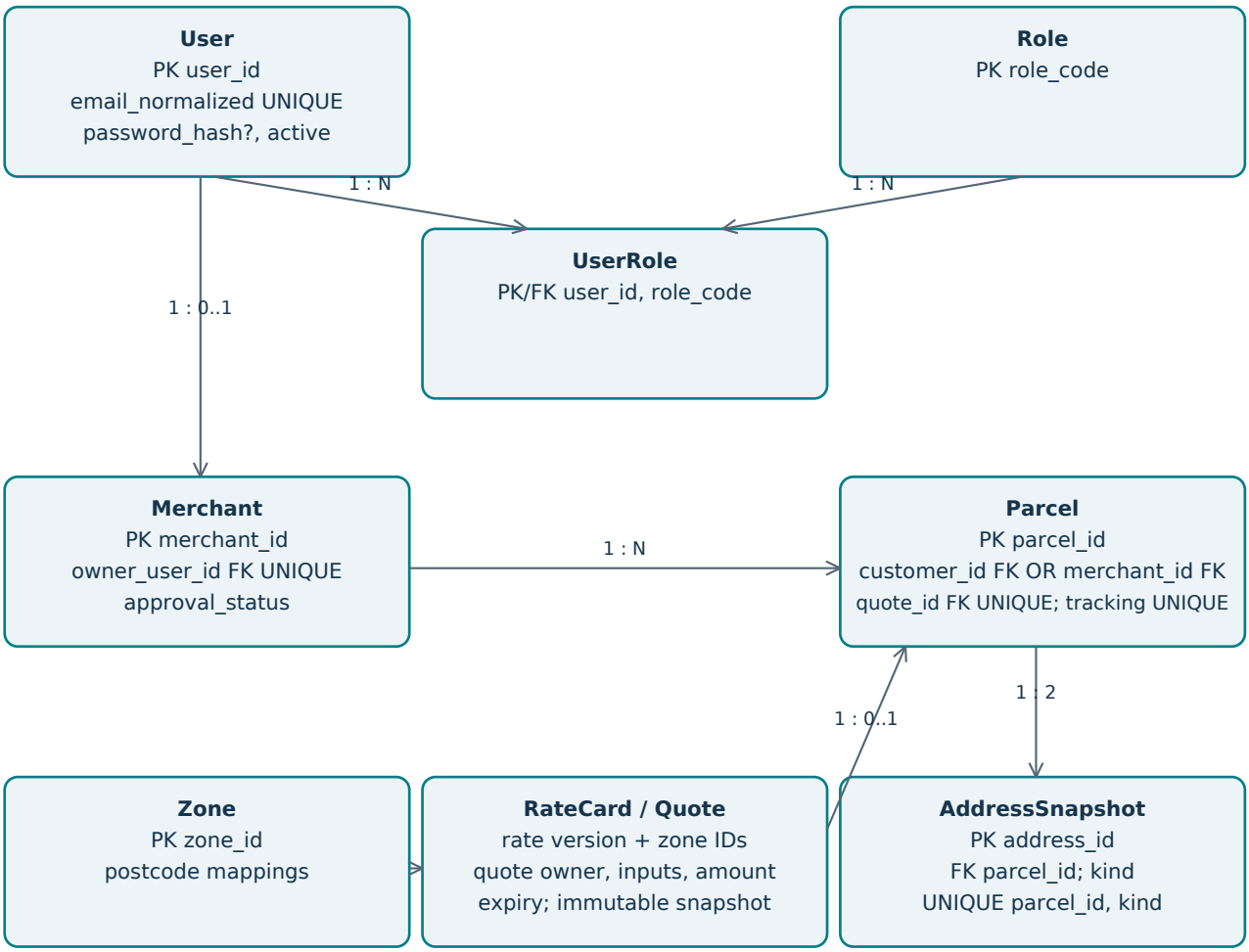


Use case	Guard and outcome
Assign courier	Authorized dispatcher; parcel eligible; courier active. At most one current assignment is saved.
Update parcel stage	Active assigned courier; legal transition; matching expected version. Stores state, history and outbox atomically.
Submit proof	Only assigned courier can attach authorized evidence. Private downloads are checked separately.
Approve merchant	Admin changes M2 business approval. A login role alone cannot bypass pending/suspended business status.

Authorization responsibilities

M1 supplies authenticated user ID, roles, account-active state and session identity. M2 resolves merchant context server-side. Every domain checks the requested resource: customer ownership, merchant ownership or current courier assignment. UI visibility improves usability but is never the security boundary.

Entity relationship diagram: accounts and booking



Cardinality labels describe logical relationships; arrows show reading direction, not data flow. Foreign keys are enforced in the shared application schema. Related business records use database foreign keys; services also validate business eligibility. User-to-Parcel (1:N customer bookings) is additional to the merchant relation shown.

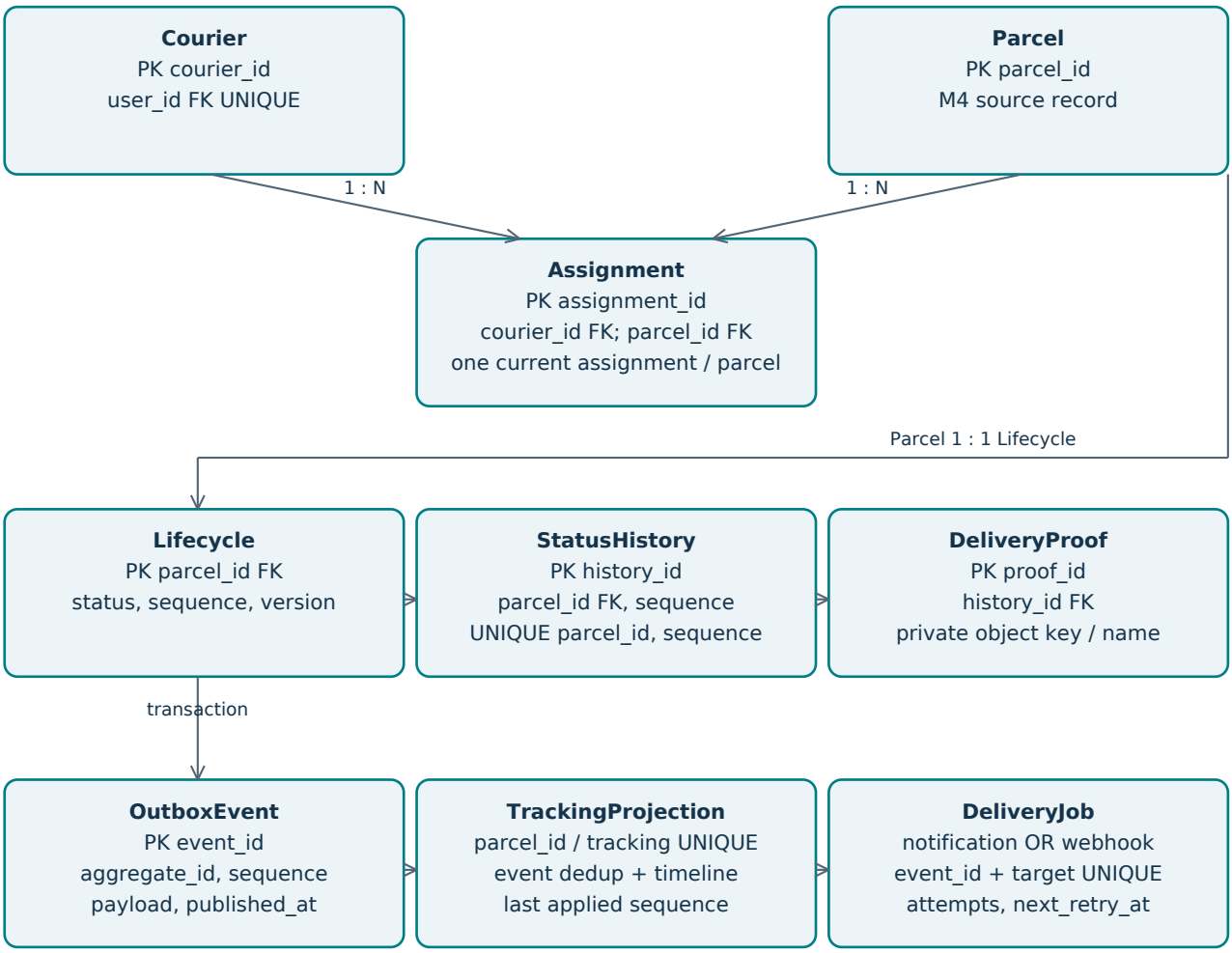
Identity supporting tables

OAuthAccount: id, user_id FK, provider, subject; UNIQUE(provider, subject). RefreshSession: id, user_id FK, family_id, token_hash UNIQUE, expires_at, revoked_at, replaced_by and session version. Password hash is null for an exclusively Google-linked account. Never store plaintext passwords or refresh tokens.

Booking invariants

Exactly one booking scope is set: customer or merchant. A parcel has one pickup and one drop-off address snapshot; enforce allowed kinds and uniqueness, and create both in the booking transaction. A quote is consumed at most once. Unique constraints cover tracking reference, quote ID, principal scope plus idempotency key, and merchant plus external reference when provided.

Entity relationship diagram: execution and integration



Lifecycle 1:N StatusHistory; StatusHistory 1:0..N DeliveryProof. Bottom arrows indicate event-derived relationships, not foreign keys. Tracking and report projections are rebuildable views, never authoritative parcel status.

Integration tables

M2: ApiCredential belongs to Merchant; key prefix enables lookup, hash enables verification. M9: Merchant 1:N WebhookSubscription; Subscription 1:N WebhookDelivery; Delivery 1:N WebhookAttempt. M8: NotificationTemplate, NotificationJob and NotificationAttempt. Both consumers maintain processed-event uniqueness.

Types, indexes and migrations

Use UUID internal IDs, UTC timestamptz, Asia/Colombo presentation, integer grams and LKR minor units. Index booking owner plus creation time, assignments by courier/current state, status history by parcel/sequence, and due retry jobs by next_retry_at. Migrations are versioned and peer-reviewed; test the complete migration history in CI. Restrict business-record deletes and use account deactivation to preserve history.

HTTP and internal interface contracts

Owner / interface	Contract and access
M1 /auth/register, /login, /refresh, /logout	POST; public entry points with validation/rate limits; refresh/logout use secure cookie and CSRF protections.
M1 GET/PATCH /me; GET/PATCH /admin/users/{id}	Self profile; admin-only account state and role administration. User listing uses GET /admin/users.
M2 /merchant/registration, /merchant/api-keys	Registration and owned credentials; administrative approval is separate.
M3 POST /quotes; GET /quotes/{id}	Signed-in principal; immutable owned quote and expiry.
M4 POST /parcels; GET /parcels; GET /parcels/{id}/receipt	Owned bookings and history; receipt requires DELIVERED.
M2 POST /merchant/quotes; POST /merchant/orders	Approved merchant API key; same pricing and booking services.
M5 POST /assignments; GET /couriers/workloads	Dispatcher/admin; parcel ID, courier ID, expectedVersion.
M6 GET /courier/jobs; POST /parcels/{id}/status	Assigned courier; nextStatus, expectedVersion, note/proof reference.
M7 GET /tracking/{number}; /tracking/{number}/events	Public-safe snapshot and SSE. Private detail requires separate authorization.
M8 /admin/notifications; M9 /merchant/webhooks	Scoped logs, controlled retry/test and configuration.

All paths have prefix /api/v1. Methods and payloads are proposed contracts to freeze in OpenAPI before implementation. List endpoints use pagination (default 50, maximum 100). Google start/callback routes follow Spring Security configuration and are documented separately.

Request and response rules

Create booking: quoteId, pickup/drop-off snapshots, parcel details and optional merchant externalReference; Idempotency-Key is required. Response 201: parcelId, trackingNumber, status and confirmedAmount. Same key and same input returns the saved result; same key with different input returns 409. Unknown client amount/role fields are rejected.

Errors use code, message, correlationId and fieldErrors. Use 400 validation, 401 authentication, 403 role denial, 404 missing or inaccessible owned record, 409 stale/conflicting state, 422 unsupported business route and 429 rate limit. Never return stack traces or secrets.

Internal ports: IdentityContext, MerchantAccess, PricingApi, BookingApi, AssignmentGuard, LifecycleApi and SenderContactQuery. Use immutable DTOs at service boundaries and keep entity access within the owning feature. In-process consumers call restricted query services. A future separate worker needs an explicit authenticated integration design.

Identity, JWT and resource security

Resolved authentication decision

The assignment explicitly requires application JWT authentication. Use short-lived access JWTs (proposed 10 minutes) held in browser memory and sent as Bearer tokens. Use opaque rotating refresh tokens in HttpOnly, Secure, SameSite cookies (proposed seven-day absolute expiry). Serve frontend/API from the same origin. Refresh returns a new access token after a page reload. Do not use localStorage for long-lived credentials.

Access claims: sub=user ID, iss, aud, iat, exp, jti and sid=session ID; roles may be included but server checks account state and authorization version. Validate signatures using an algorithm allowlist. Read active account/session state on protected requests so logout, suspension and role changes take effect without waiting for JWT expiry. This trades fully stateless JWT validation for immediate revocation.

Session and Google controls

Hash passwords using an established adaptive password encoder. Hash stored refresh tokens; rotate atomically and revoke the token family on confirmed reuse. Serialize browser refresh calls to avoid accidental concurrent reuse. Logout revokes the session and clears cookies. CSRF protection applies to refresh/logout and other cookie-authenticated mutations; restrict CORS and check origins.

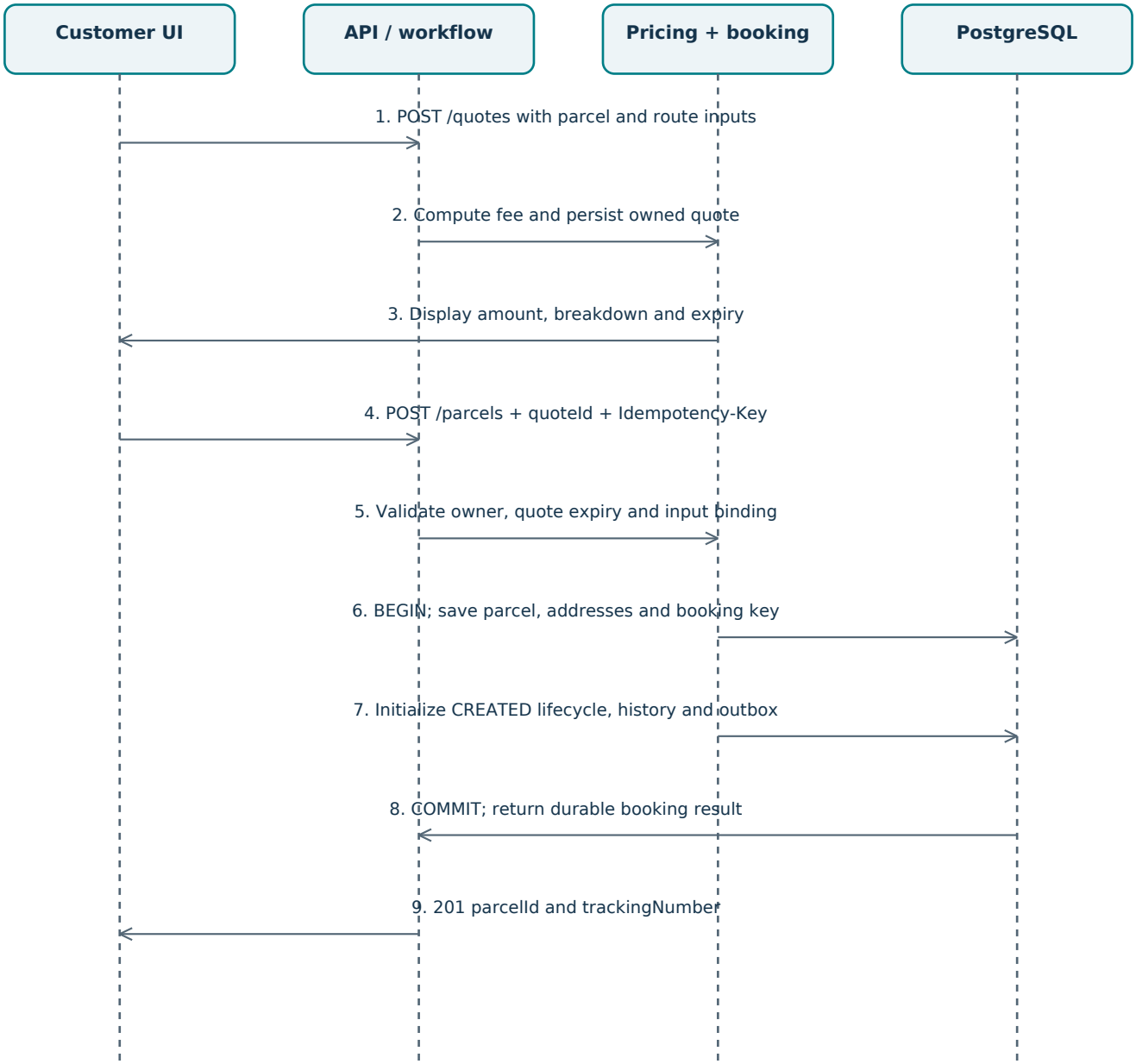
Google login uses Authorization Code with PKCE, state and nonce. Verify ID-token issuer, audience, signature and expiry through the security library. Identify users by provider plus subject. Never grant staff roles from provider attributes or automatically link accounts solely because email strings match. Google callback establishes a refresh session; the UI obtains an application JWT through refresh, not a token in a redirect URL.

Actor	Allowed scope
CUSTOMER	Own profile, quotes, parcels and receipts.
MERCHANT	Approved own-business data after server-side membership resolution.
COURIER	Own jobs and currently assigned parcel updates; no global customer search.
DISPATCHER_ADMIN	Operational administration with audit logging; no secret disclosure.
Anonymous	Signup/login and sanitized public tracking only.

Protection of data and integrations

Store proof privately, validate uploads and authorize each download. Redact addresses, phones, credentials and tokens in logs. Use TLS outside localhost. Hash API keys; encrypt retrievable webhook signing secrets with a separately managed key. Apply rate limits to login, refresh and public tracking. Production refuses development authentication profiles.

Sequence: customer creates a delivery



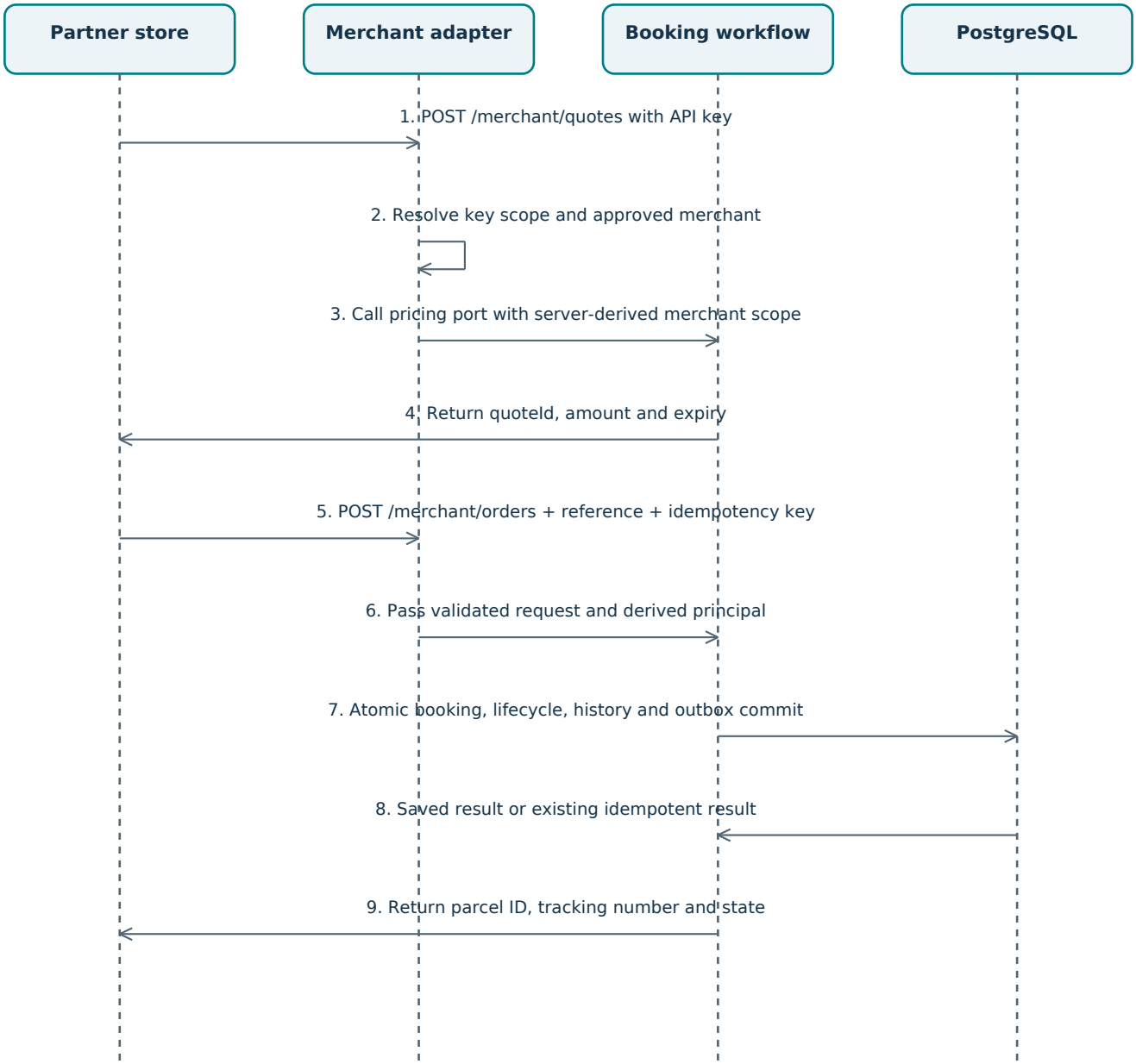
Transaction boundary

M4 coordinates public M3 and M6 interfaces using the same datasource and transaction manager. M6 failure rolls back the parcel, address snapshots, quote consumption, history and outbox together. Nothing is published before commit. The outbox relay and projections run afterward in bounded background consumers.

Alternative paths

Expired or modified quote: return 409 and offer recalculation. Duplicate request: return the prior booking. Unsupported route: explain service coverage. Lost HTTP response after commit: retry the same idempotency key. A temporarily absent tracking projection shows catching up rather than claiming the booking failed.

Sequence: merchant API order creation



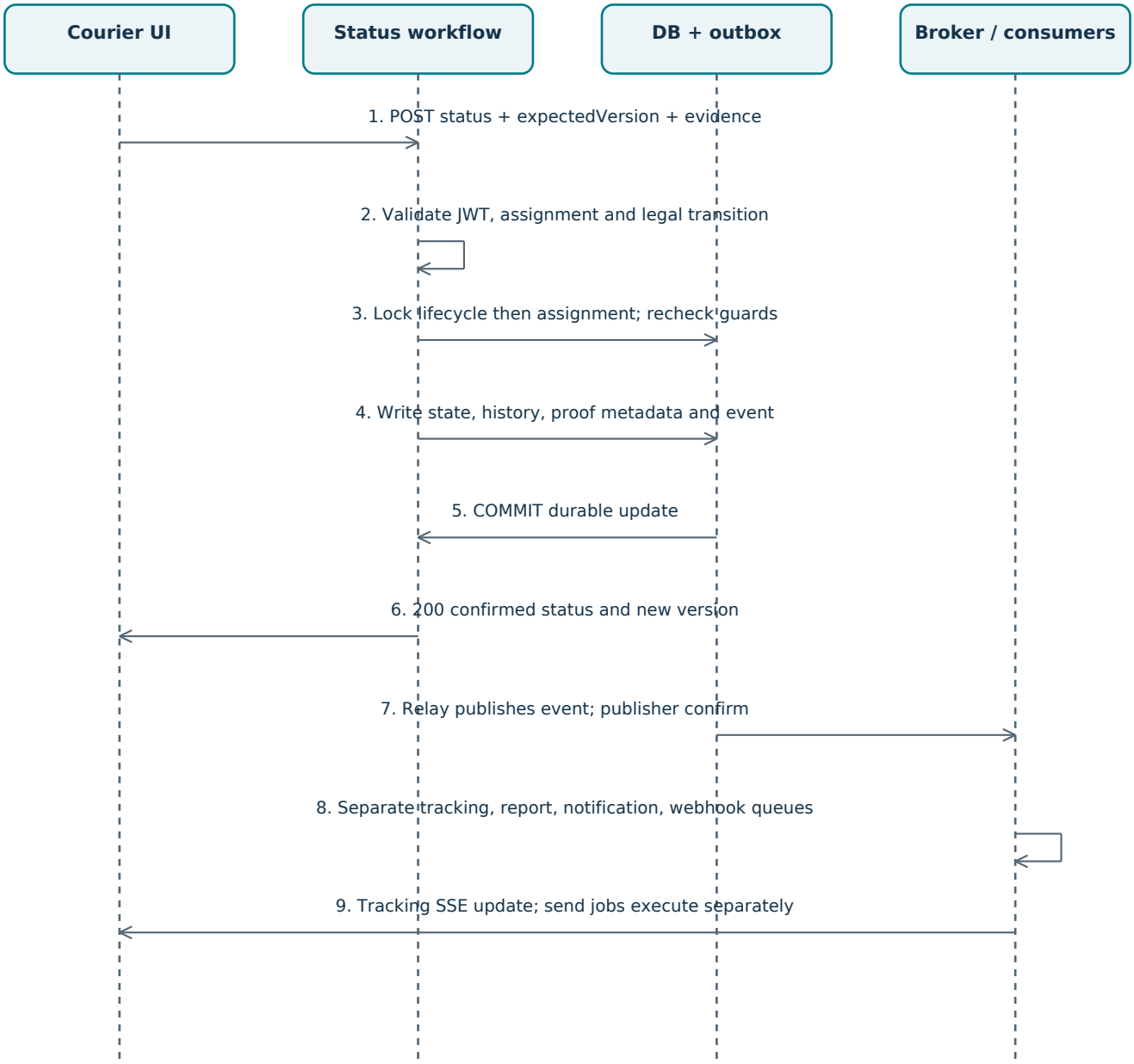
Reuse and tenant isolation

M2 is an inbound adapter; it does not duplicate M4 booking logic. A merchantId sent in the request body never determines authorization. The credential resolves the merchant and scopes; approval and revocation are checked at request time. Bound externalReference to the authenticated merchant.

Failure behaviour

Invalid key returns 401; revoked scope or inactive business is denied. Duplicate external reference with incompatible input returns 409. A provider timeout never warrants an automatic new reference or new idempotency key. Subsequent M6 state events trigger M9 webhook delivery independently of this request.

Sequence: courier update and asynchronous fan-out



Concurrency and failure

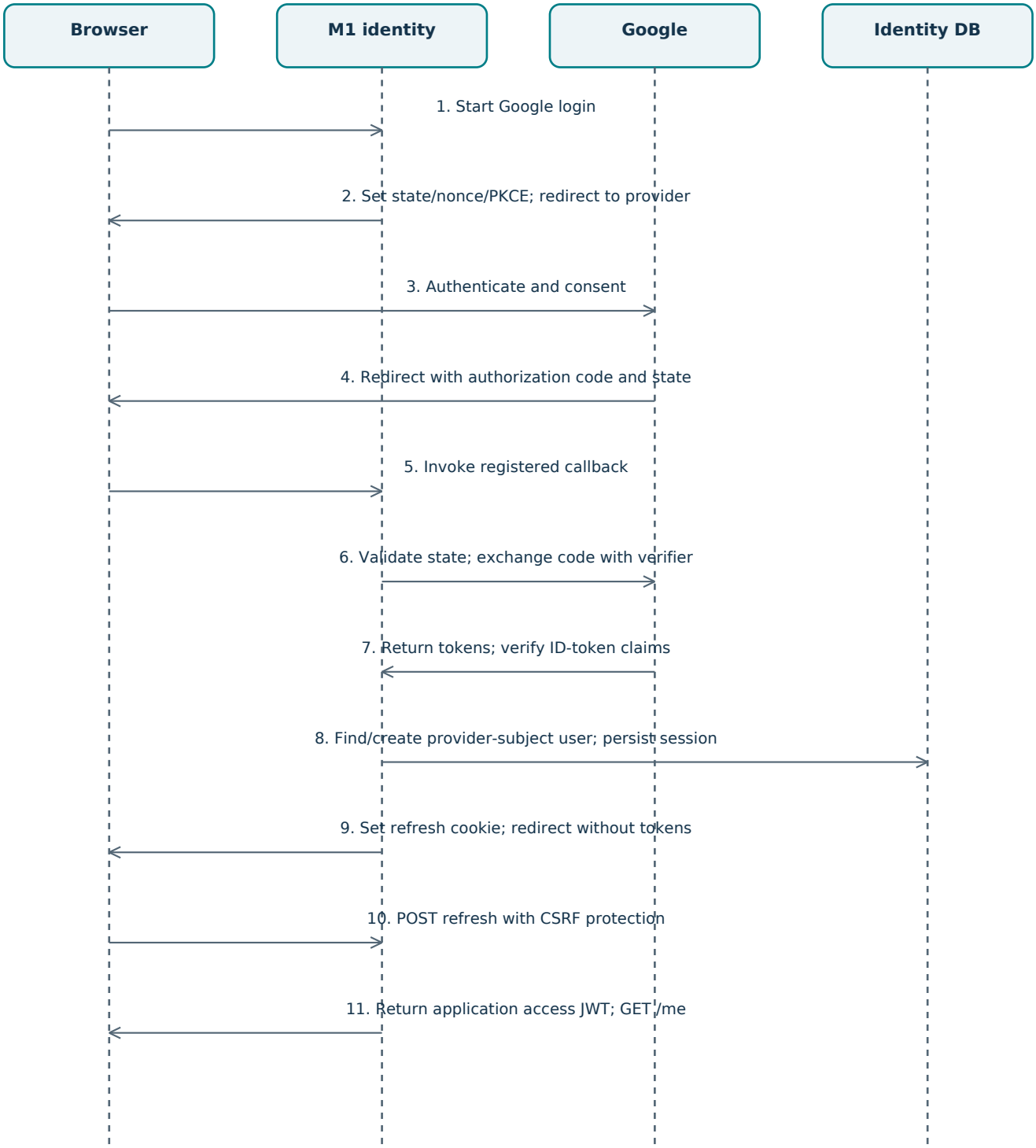
Assignment and status workflows acquire locks in the same order: lifecycle before current assignment. A partial unique index prevents two current assignments. A stale expectedVersion returns 409 with safe current state. Courier ownership is checked inside the protected transaction, not only before it.

Broker outage leaves an unpublished outbox row for retry. A crash after publish but before marking sent can duplicate events; consumers deduplicate. Notification or webhook failure never rolls back the courier update. The last arrow summarizes independent downstream effects, not a broker connection to the browser.

Lifecycle guard

CREATED -> PICKED_UP -> IN_TRANSIT -> OUT_FOR_DELIVERY -> DELIVERED or FAILED. Failed delivery requires a reason. Success requires configured proof. Terminal changes and retries are not silently permitted in this baseline. An initial assignment is required before courier progress begins.

Sequence: Google OAuth / OIDC login



Failure and linking policy

Reject state/nonce mismatch, expired code, invalid issuer/audience or signature. A denied consent returns to login with a safe message. If a verified Google email matches an existing local account but no provider link exists, require authenticated explicit linking; do not merge accounts automatically.

Google OAuth client secrets stay on the server. Register exact callback URLs for local and deployed environments. Mock-provider tests support independent M1 development, but the assignment demonstration must include real Google login. A provider ID token is not the application JWT.

Events, notifications and webhook delivery

Pattern	Application and rationale
Request/reply	React and merchant REST requests return immediate validation and committed booking/status outcomes.
Publish/subscribe via broker	Status events fan out to separate queues for tracking, operations, notifications and webhooks; one shared queue would load-balance instead.
Transactional outbox	Persist event with business state; relay publishes and records confirmation.
Idempotent consumer	Unique event receipt / target key prevents duplicate timeline rows, counters and logical jobs.
Scheduled retry / dead letter	Persist attempts and next retry time; exhausted jobs remain visible for controlled replay.

Versioned event contract

ParcelStatusChanged.v1 contains eventId, occurredAt, parcelId, trackingNumber, merchantId (nullable), externalReference (nullable), sequence, previousStatus, status and schemaVersion. Do not include addresses, phones, photos or private notes. Event schema changes must remain compatible or use a new version.

Use durable topic exchange swiftdrop.events with routing keys parcel.created.v1 and parcel.status.changed.v1. Bind independent durable queues tracking.events.v1, operations.events.v1, notifications.status.v1 and webhooks.status.v1 as appropriate. Messages are persistent; acknowledge only after durable consumer work. Bound worker concurrency and monitor oldest pending work.

Baseline retry policy

Notification: three total attempts with delays of 10 and 60 seconds. Webhooks: five total attempts with delays of 10, 30, 120 and 600 seconds, plus jitter; 5-second HTTP timeout. Non-2xx and timeout responses are failures. These are student-release policies, superseding the longer enterprise proposal. Dead-letter/manual replay preserves the original event ID and complete attempt history.

Webhook security and receiver

Send JSON with X-SwiftDrop-Event-Id, X-SwiftDrop-Timestamp and X-SwiftDrop-Signature. Signature = hex HMAC-SHA256(secret, timestamp + "." + exact UTF-8 body). Refresh timestamp/signature per attempt; preserve eventId. Receiver checks constant-time signature equality, a proposed five-minute clock window and event deduplication.

Allow public HTTPS destinations only; block private, loopback, link-local and metadata ranges; validate DNS and disable redirects. Local demo receiver is an exact dev-only allowlist exception. Signatures use a secret distinct from inbound API keys. Log response status and safe diagnostics, not arbitrary sensitive bodies.

UI structure and experience design

Role / route area	Screens and important states
Public	Landing, login/signup, Google callback result, tracking lookup and timeline.
Customer	New delivery: addresses -> parcel -> quote review -> confirmation; history and receipt.
Merchant	Approval status, orders, API keys, webhook settings and delivery attempts.
Courier	Today's tasks, parcel details, next legal action, proof form and failure reason.
Dispatcher/Admin	Unassigned list, assignment panel, workloads, reports, user and merchant management, zones/rates.

Shared interaction rules

Use one application shell with feature route registration. Keep action labels and status names consistent. Preserve valid input on errors; show field-level feedback. Separate empty, loading, unauthorized, failed and catching-up states. Label public tracking as Tracking / waybill number. Never use colour as the only status indicator.

Member 1 screen specification

Login: email, password, show/hide control, submit and Google button; generic invalid-credential message. Signup: name, email, password and confirmation; public role is always CUSTOMER. Profile: read-only identity fields and editable allowed contact fields. Admin users: paginated list, active state, role editor and audited suspension. Do not expose password hashes, refresh tokens or API keys.

Responsive and accessible design

Target WCAG 2.2 AA with keyboard navigation, visible focus, labels, sufficient contrast and screen-reader status announcements. Courier/public tracking screens must work at 360 CSS pixels without horizontal scrolling. Use 44-pixel primary touch targets as a product choice. Core merchant functions should be reachable within three navigation actions; necessary form steps and safety checks are not constrained to three clicks.

Tracking behaviour

SSE sends public-safe snapshots and sequence IDs. Reconnect with last event ID or refetch the snapshot when history is unavailable. Display last updated time and connection state. Private streams use authenticated fetch streaming; never put access JWTs in URLs. Route SSE connections to shared committed projections so multiple API instances can recover state.

Performance design targets

Student-demo target: 50 active users, 20 requests/second for ten minutes, API p95 under one second and committed state visible through SSE within five seconds at p95. These targets require measurement and are not achieved results. The enterprise SRS's larger workload and 99.9% availability are later validation targets, not guarantees from local Compose.

M1 - Identity and Access (leader)

Story	Implementation and acceptance
IAM-01 Customer signup	React form -> registration service -> users/roles. Valid signup creates CUSTOMER only; duplicate/invalid email rejected.
IAM-02 Login and logout	Password encoder, JWT issuer and refresh session. Wrong password denied; logout revokes session and clears cookie.
IAM-03 Google login	OIDC adapter and oauth_accounts. State/nonce/claims validated; real Google demo plus mocked automated cases.
IAM-04 Contact profile	GET/PATCH /me. Allowlisted fields only; cannot change own role or active state.
IAM-05 User administration	Admin screen and active-state service. Suspension invalidates authorization on subsequent protected requests.
IAM-06 Roles and ownership	Role administration + shared IdentityContext. Public requests cannot self-elevate; each domain supplies ownership checks.
IAM-07 Session renewal	Atomic refresh rotation, expiry, family revocation and concurrent-refresh handling. Old/reused tokens fail.

Component responsibilities

AuthController validates DTOs; AuthenticationService handles credentials and sessions; GoogleIdentityAdapter maps verified external identity; TokenService issues/validates application JWTs; UserService handles allowed profile/admin changes. Repositories belong only to identity. React auth state stores access JWT in memory, bootstraps with refresh and obtains /me. A request wrapper handles one serialized refresh attempt and never loops indefinitely.

Contract provided to the team

IdentityContext exposes userId, roles, active, sessionId and authorizationVersion. M2 supplies approved merchant scope through its own port. Publish role names, example 401/403 errors, test principals and JWT validation configuration early. M1 must not implement dispatch logic, merchant approval or business-record ownership on behalf of all modules.

Development order

1. Establish and verify the shared repository baseline. 2. Freeze identity contracts and migrations. 3. Implement signup/login/me/logout end to end. 4. Add Google. 5. Add profile and user administration. 6. Add rotation/revocation tests. 7. Replace one module's fake principal with real identity and test allowed and forbidden access. Use a feature branch and a peer reviewer. This is one of nine equally important feature workstreams; the leader does not implement everyone else's feature.

Verification, release and source register

Test area	Required evidence
Authorization	Customer A cannot read B receipts; courier A cannot update B assignments; merchant A cannot see B logs; suspended account denied.
Transactions and races	Failed lifecycle initialization rolls back booking; concurrent assignments leave one active row; stale version returns 409.
Event integration	Real RabbitMQ status-to-consumer test; broker outage recovery; duplicate and out-of-order events; webhook 500 then 200.
Identity	Password and Google paths; JWT expiry; refresh reuse; logout; public role injection; CSRF and forbidden profile fields.
UI and performance	Mobile/keyboard checks, error recovery, SSE reconnect and measured workload results.
Delivery	Fresh Compose startup, real Google credentials configured privately, deployed demo, source README and actual coverage report.

Release gates and contributions

First integrate login -> quote -> booking -> CREATED tracking. Then assignment -> courier stages -> SSE -> notification/webhook. Complete receipts and reports; test failures and permissions before adding optional features. Each member records story IDs, commits, schema changes, tests and a short demo. Coverage is measured; a suggested 80% business-service target is not a brief-mandated threshold.

Open decisions before implementation freeze

Confirm nine-person approval, deadline and hosting; actual rate/service rules; Google OAuth project and callback URLs; required proof policy; retention periods; production secret storage and backups. Use pinned compatible dependencies after verifying the starter on team machines. This design does not certify the earlier starter as matching every contract without inspection.

Source register and interpretation

[S1] Group Assignment(1).pdf, four pages, supplied university brief; accessed 31 October 2026. Authoritative functional requirements, design deliverables, JWT/Google, broker, Compose, testing and demo obligations.

[S2] SwiftDrop_Nine_Member_System_Blueprint.pdf, 20 pages, revision saved 27 October 2026 at 05:22 UTC. Feature ownership and seven IAM stories retained. Version 2 simplifies the earlier database and runtime separation into one layered application. Leader owns M1.

[S3] Competitive audit and SRS in this conversation, 29 October 2026. Used for proposed UX and resilience improvements. Enterprise extensions and targets are explicitly separated from the assignment baseline. No authenticated competitor audit or performance benchmark is claimed.

Technical references previously consulted: RabbitMQ Reliability Guide (https://www.rabbitmq.com/docs/reliability); OAuth security BCP RFC 9700 (https://www.rfc-editor.org/rfc/rfc9700.html); WCAG 2.2 (https://www.w3.org/TR/WCAG22/); Spring transaction and repository references are listed on page 31. Implementation should check documentation for pinned dependency versions.

Document status: proposed design baseline for team review. Diagrams express intended behaviour, not evidence of completed code, test success or production readiness.

M2 - Merchant accounts and order API

Story	User goal	Minimum acceptance
MER-01	Register business	Business form saves a PENDING application linked to the signed-in owner.
MER-02	Approve merchant	Admin can approve/reject with reason; pending/suspended businesses cannot create API orders.
MER-03	Issue API credential	Show a scoped key once; store only prefix/hash; revoked key fails immediately.
MER-04	Request merchant quote	API key resolves business scope and invokes the shared pricing service.
MER-05	Create API order	Reuse booking service with quote, reference and idempotency key; repeated input yields one parcel.
MER-06	View business orders	Paginated dashboard filters only the authenticated merchant's orders.
MER-07	Revoke and audit access	Owner can revoke a key; safe audit records omit keys and request bodies.

Screens and Java classes

MerchantRegistrationPage, MerchantDashboard and ApiKeySettings; admin approval queue. Backend: MerchantController, MerchantService, MerchantApiKeyFilter/adapter and MerchantRepository. Identity resolves the user; M2 resolves approved merchant scope.

Tables and constraints

merchants(id, owner_user_id FK UNIQUE, name, approval_status, reviewed_by FK, reviewed_at); api_credentials(id, merchant_id FK, prefix, secret_hash, scopes, revoked_at); api_request_audit with sanitized metadata.

Dependencies and independent development

Uses M1 identity, M3 quote service and M4 booking service. Mock those services while testing M2. Provide MerchantAccessService to M4/M9; never duplicate parcel creation.

Tests and demonstration

Demo pending denial, admin approval, key creation, one order via an HTTP client, duplicate retry and revocation. Test merchant A/B isolation and role versus approval-state differences.

M3 - Zones and pricing

Story	User goal	Minimum acceptance
PRI-01	Maintain zones	Admin defines supported areas and postcode mappings; duplicate mappings rejected.
PRI-02	Maintain rate cards	Admin enters route base, included weight, extra-weight rate and size surcharge with an effective version.
PRI-03	Calculate estimate	Validated route/size/grams produce deterministic amount in LKR minor units.
PRI-04	Reject unsupported inputs	Unsupported zone pair, inactive rate, zero/negative weight and excessive weight receive actionable errors.
PRI-05	Save quote	Persist authenticated owner scope, input fingerprint, amount, rate version and expiry.
PRI-06	Validate quote at booking	Reject wrong owner, expiry or changed price-driving inputs.
PRI-07	Inspect quote breakdown	UI shows base, additional weight, size charge, currency and expiry; no client amount overrides.

Screens and Java classes

ZoneEditor, RateCardEditor and QuoteForm. Backend: PricingController, PricingService, PricingStrategy, ZoneRepository, RateCardRepository and QuoteRepository. The service selects a rate; the strategy calculates the amount.

Tables and constraints

zones; zone_postcodes FK zone; rate_cards with pickup_zone_id/dropoff_zone_id FKs and version; quotations with scope, input_hash, amount_minor, currency, rate version and expires_at. Historical quotes retain snapshots.

Dependencies and independent development

Uses authenticated principal and a replaceable clock. M4 and M2 call the same PricingService. Demo fixtures provide two zones and fixed rates; they are not actual company prices.

Tests and demonstration

Test weight rounding at 999/1000/1001 grams, sizes, absent routes, rate changes, expiry and foreign quote access. Formula: base + max(0, ceil(grams/1000)-includedKg)*extraKgRate + sizeSurcharge.

M4 - Booking, history and receipts

Story	User goal	Minimum acceptance
BKG-01	Enter delivery details	Form validates pickup/drop-off contact and parcel inputs without losing valid entries.
BKG-02	Review quote	Display owned fee and expiry before confirmation; changed inputs require a new quote.
BKG-03	Confirm booking	One transaction creates addresses, parcel, lifecycle, history and outbox; return tracking reference.
BKG-04	Retry safely	Same principal/key/payload returns prior result; different payload with same key returns 409.
BKG-05	View delivery history	Paginated customer-owned list supports status/date filters.
BKG-06	Download receipt	Only owner can download delivery receipt after DELIVERED; it is not a payment receipt.
BKG-07	View booking confirmation	Show reference, amount and next step; allow tracking projection to catch up.

Screens and Java classes

BookingWizard, ConfirmationPage, DeliveryHistory and ReceiptButton. Backend: ParcelController, BookingService, ReceiptService, ParcelRepository and AddressRepository. BookingService is the transaction coordinator.

Tables and constraints

parcels with customer_id XOR merchant_id, quote_id FK UNIQUE and tracking_number UNIQUE; parcel_addresses with parcel/kind UNIQUE; booking_requests with principal/idempotency UNIQUE and request hash; receipts; booking outbox.

Dependencies and independent development

Calls M3 quote validation and M6 lifecycle initialization in the same transaction. M2 calls this service for API orders. M7 receives created events. Provide a minimal sender-contact query to M8.

Tests and demonstration

Test rollback if lifecycle initialization fails, lost-response retries, owner/merchant separation, expired quote, duplicate external reference and receipt access. Begin with recipient-name proof support; photo storage is optional.

M5 - Dispatch, workload and reports

Story	User goal	Minimum acceptance
DSP-01	Manage courier work profiles	Admin creates profile for an existing courier-role user; unique user/profile link.
DSP-02	List unassigned parcels	Show eligible CREATED parcels with basic route and age; exclude already assigned work.
DSP-03	Assign delivery	Select eligible active courier and atomically create one current assignment.
DSP-04	Reassign safely	Before pickup, admin may reassign with reason; use the same locks/version rules as courier updates.
DSP-05	Monitor workloads	Show current active assignments; terminal jobs are excluded.
DSP-06	Daily delivery report	Count delivered/failed parcels by event date in Asia/Colombo; state the denominator.
DSP-07	Inspect operational exceptions	Highlight stale/unassigned/failed parcels with safe detail and next administrative action.

Screens and Java classes

DispatchBoard, CourierWorkload, CourierProfileEditor and DailyReport. Backend: DispatchController, AssignmentService, CourierService, ReportService and their repositories. Default before-pickup reassignment only; later reassignment requires new rules.

Tables and constraints

couriers(id,user_id FK UNIQUE,active,service_zone_id FK); assignments(parcel_id FK,courier_id FK,assigned_at,ended_at); partial UNIQUE(parcel_id) WHERE ended_at IS NULL; operational projection and processed event IDs.

Dependencies and independent development

Calls M1 account lookup, M4 parcel-summary service and M6 lifecycle guard. Assignment/status workflows lock in the same order. Reports consume lifecycle events or use an explicitly approved reporting query, not ad-hoc frontend joins.

Tests and demonstration

Test two dispatchers assigning the same parcel, inactive courier, reassignment/status race and duplicate event counters. Success rate = delivered/(delivered+failed) for terminal events in the chosen period; show N/A when denominator is zero.

M6 - Courier execution and lifecycle

Story	User goal	Minimum acceptance
EXE-01	View today's tasks	Courier sees only current assigned work, including carried-over active tasks.
EXE-02	Inspect delivery detail	Authorized task shows required contact and addresses; no global parcel enumeration.
EXE-03	Update progress	Only the next allowed state is accepted; expectedVersion protects concurrent changes.
EXE-04	Complete delivery	Required recipient name or configured proof is persisted with DELIVERED.
EXE-05	Record failure	FAILED requires reason; terminal status cannot silently reopen.
EXE-06	Read status history	Authorized history is ordered by sequence with actor and occurrence time.
EXE-07	Publish durable event	Every committed lifecycle change has an outbox event; broker failure does not lose it.

Screens and Java classes

CourierTaskList, TaskDetail, StatusAction and Proof/Failure forms. Backend: CourierController, LifecycleService, ProofService, LifecycleRepository, StatusHistoryRepository and OutboxRepository. M6 is the single writer of canonical status.

Tables and constraints

parcel_lifecycle(parcel_id PK/FK,status,sequence,version); status_history(parcel_id FK,sequence UNIQUE per parcel,actor_id FK,reason,time); delivery_proofs(history_id FK,recipient_name,private_object_key); execution outbox.

Dependencies and independent development

M4 calls initializeLifecycle during booking. M5 provides assignment guard. M7/M8/M9 consume events. Use mock assignment and parcel services for isolated tests; keep authorization tests against real security as a separate suite.

Tests and demonstration

Test illegal jumps, wrong courier, stale version, missing failure reason, missing proof, database rollback and broker outage. Photo upload: validate size/type, private storage and authorized download; compensate abandoned uploads with cleanup.

M7 - Tracking and live updates

Story	User goal	Minimum acceptance
TRK-01	Public tracking lookup	High-entropy tracking number returns public status; unknown input is safe.
TRK-02	Display timeline	Order by parcel sequence; duplicate events create no duplicate rows.
TRK-03	Stream live changes	Real committed event reaches SSE and updates an open screen.
TRK-04	Mobile tracking	Lookup and timeline work at 360px; status has text, not colour alone.
TRK-05	Authenticated tracking detail	Owner-only details require ownership checks independent of public lookup.
TRK-06	Reconnect safely	Resume from sequence or refetch; late events never regress status.
TRK-07	Protect public data	Omit addresses, phone numbers, recipient names, notes and proof; rate-limit lookups.

Screens and Java classes

TrackingSearch, Timeline and ConnectionStatus. Backend: TrackingController, TrackingQueryService, TrackingEventConsumer and SseConnectionRegistry. Consumers update persisted state; the registry only manages live connections.

Tables and constraints

tracking_snapshots(parcel_id UNIQUE,tracking_number UNIQUE,status,last_sequence,updated_at); tracking_timeline(event_id UNIQUE,parcel_id,sequence UNIQUE per parcel); processed_events. These are projections, not sources of truth.

Dependencies and independent development

Consumes M4/M6 events; uses M1/M4 only for private detail. For incomplete dependencies, inject fixed event fixtures. In the first release one backend instance hosts SSE; multi-replica streaming requires shared distribution design before scaling.

Tests and demonstration

Demo sequence 1,3,2,3: latest stays 3; timeline can fill 2 once. Disconnect/reconnect and test missing replay history. Measure commit-to-screen delay, excluding an offline courier's unsubmitted work.

M8 - Notifications and logs

Story	User goal	Minimum acceptance
NOT-01	Consume status event	A real broker event creates one persisted notification job.
NOT-02	Maintain message templates	Admin edits allowed placeholders and previews synthetic content.
NOT-03	Resolve recipient safely	Query the authorized saved sender contact; do not trust an arbitrary event recipient.
NOT-04	View delivery logs	Admin sees pending/sent/failed states with masked contact and safe error.
NOT-05	Retry provider failures	Persist attempt count and next retry; stop after configured maximum.
NOT-06	Deduplicate jobs	Repeated event/recipient/channel yields one logical job; provider receives stable idempotency key.
NOT-07	Replay after repair	Admin can retry an exhausted job; attempt history and original event remain intact.

Screens and Java classes

NotificationTemplateEditor, NotificationLog and AttemptDetails. Backend: NotificationConsumer, NotificationService, TemplateService, SmsGateway adapter and job/attempt repositories. Start with a mock SMS gateway, explicitly allowed in the brief.

Tables and constraints

notification_templates; notification_jobs(event_id,target,channel UNIQUE,rendered_body,state,next_retry_at); notification_attempts(job_id FK,attempt_no,outcome); processed_events. Restrict message content visibility and retention.

Dependencies and independent development

Consumes its own RabbitMQ queue and uses M4 SenderContactQuery. M8 coordinates exchange/queue configuration and outbox relay plumbing; each owner still writes their own events and tests.

Tests and demonstration

Test duplicate ingestion, missing contact, temporary failure then success, exhaustion and admin retry. Distinguish provider acceptance from actual user delivery. Without provider idempotency, an ambiguous timeout can cause duplicate sends; document this limit.

M9 - Merchant webhooks and diagnostics

Story	User goal	Minimum acceptance
WHK-01	Register callback	Approved merchant saves a validated HTTPS endpoint under its own business.
WHK-02	Select event subscriptions	Only supported stages; no unrelated business or customer-only order callbacks.
WHK-03	Sign outgoing request	Timestamp/body HMAC and immutable event ID verify source and support deduplication.
WHK-04	Retry delivery	Timeout/non-2xx schedules durable retry; exhausted status visible.
WHK-05	Inspect attempts	Merchant sees only owned sanitized attempt logs and next retry time.
WHK-06	Send test callback	Clearly marked synthetic event is signed, delivered and logged.
WHK-07	Disable/rotate secret	Disabled endpoints stop pending sends; explicit versioned secret cutover policy.

Screens and Java classes

WebhookSettings, DeliveryLog and AttemptDetail. Backend: WebhookConsumer, SubscriptionService, DeliveryService, HttpWebhookClient and repositories. Include a small demo merchant receiver that verifies signatures and can deliberately fail.

Tables and constraints

webhook_subscriptions(merchant_id FK,url,encrypted_secret,secret_version,enabled); webhook_deliveries(subscription_id FK,event_id UNIQUE per subscription,payload,state,next_retry_at); webhook_attempts(delivery_id FK,attempt_no,response_code); processed_events.

Dependencies and independent development

Uses M2 approved merchant scope and consumes M6 events. API keys are not signing secrets. Resolve approved target at each send; block SSRF destinations and redirects. A demo localhost receiver is allowed only by explicit development configuration.

Tests and demonstration

Test signature mismatch, stale timestamp, duplicate ID, cross-merchant read, private-IP URL rejection, endpoint 500 then 200, timeout, disable and rotation. Preserve eventId on replay and redact secrets; do not promise exactly-once delivery.

Understand the layers through one example

Example: a customer clicks Confirm delivery. The customer must not be able to choose an arbitrary price or create a parcel belonging to somebody else. Each layer has one clear job.

Step / layer	What happens	What must not happen
1 React	Collect inputs and send a request DTO as JSON with the access JWT.	Treat a price calculated in JavaScript as authoritative.
2 Security filter	Verify JWT and active session; establish authenticated user.	Trust userId or role supplied in a request body.
3 ParcelController	Validate field types/required values and call BookingService.	Calculate prices, issue SQL or send webhooks.
4 BookingService	Check quote owner/expiry; apply business rules; start transaction; call lifecycle service.	Return success before the transaction commits.
5 Repositories	Persist parcel, address snapshots, history and outbox through JPA.	Decide HTTP response status or render UI.
6 PostgreSQL	Enforce FKs, uniqueness and atomic commit/rollback.	Replace application authorization checks.
7 Response	Service maps saved result to response DTO; controller returns JSON; UI shows confirmation.	Expose database entities containing private fields.

Vocabulary in simple language

DTO: the agreed shape of information entering or leaving an API. Entity: a Java representation of a stored record. Repository: code responsible for reading/writing those records. Service: code that decides what the business permits. Controller: the HTTP entry point. Transaction: a group of database changes that commit together or roll back together.

Spring annotations and dependency injection

@RestController marks HTTP handlers; @Service marks business components; a JpaRepository interface provides persistence operations; @Entity maps a record; @Valid checks input constraints. Use constructor injection so dependencies are explicit. @Transactional belongs on an externally invoked Spring service operation; self-calling a transactional method may bypass proxy interception. Test rollback explicitly. See [T1-T3], page 31.

Business rules versus field validation

A positive weight is input validation. A supported route, unexpired owned quote and eligible courier are business rules. A UNIQUE tracking number is a database constraint. You often need all three checks; placing everything in one controller makes testing and maintenance difficult.

Java project structure and shared standards

Use one Git repository. Package by layer, with a feature subpackage in each layer. Every teammate owns matching files across layers. This is the agreed layered architecture expressed directly in folder names.

Path under repository	Purpose / owner
frontend/src/app/	Shared navigation, routing, layout and authentication bootstrap.
frontend/src/features/identity/ ... webhook/	Nine feature areas; screens, components and API calls.
backend/src/main/java/com/swiftdrop/controller/<feature>/	HTTP handlers. Controllers depend on services only.
.../service/<feature>/	Business logic and transaction coordinators; cross-feature service calls use published contracts.
.../repository/<feature>/	Spring Data repositories and explicit persistence queries.
.../entity/<feature>/	JPA entities and enums, owned by the feature.
.../dto/<feature>/	Request/response DTOs; do not reuse JPA entities as HTTP contracts.
.../security/, config/, exception/, integration/	Security filters, configuration, safe errors, broker/provider adapters.
backend/src/main/resources/db/migration/	One ordered Flyway migration history. Coordinate versions before merging.
backend/src/test/java/com/swiftdrop/	Unit, security, repository, API and integration tests.
contracts/; docs/; compose.yaml	OpenAPI/event examples, designs, contribution records and deployment.

Migration policy

Use one migration sequence, assigned during pull-request review; never modify an already applied migration. A new change gets a new migration. All team databases use the complete schema; mocks replace unfinished behaviour, not missing FK target tables. Keep synthetic seed data separate from production migrations. Use PostgreSQL in integration tests, not only an in-memory substitute.

Common rules

Use consistent DTO naming, UTC timestamps, LKR minor units, integer grams, UUID IDs and safe error codes. Paginate list endpoints. No secrets in Git, no direct browser database access, no controllers calling repositories and no circular service dependencies. Keep the constructor dependency graph small. Add interfaces where replacement/testing helps; do not create abstractions without a purpose.

The earlier starter pack is a reference scaffold, not proof that this revised structure is already present. Inspect and adapt it once in the shared baseline branch before distributing work. Freeze the Java/Boot versions and verified startup/test commands in README and AGENTS.md.

Beginner implementation plan and team workflow

Checkpoint	Work to complete	Exit condition
A - Shared foundation	Create repository; agree Java/Boot version; scaffold layers, React shell, DB, migration and Compose.	All nine can run the same health screen and database-backed sample.
B - Contract agreement	Roles, lifecycle, IDs, DTOs, errors, service signatures and event examples; assign reviewers.	Two customers/merchants/couriers and one admin fixture; contracts are committed.
C - First vertical slice	M1 signup/login; M3 quote; M4 booking; M6 CREATED; M7 snapshot.	User can log in, quote, book and track one real parcel.
D - Operations	M5 assignment; M6 progression/proof/failure; M7 SSE; M8 consumer.	Dispatcher-to-courier-to-tracking demo with real RabbitMQ.
E - Merchant integration	M2 approval/key/orders; M9 signed callback and retries.	External demo store creates one order and receives a verified callback.
F - Finish and verify	Receipts, reports, admin screens, Google, failure tests, mobile checks, docs and deployment.	Fresh startup; assignment demo; actual test/coverage and contributions.

These are dependency checkpoints, not promised calendar dates. Set dates after confirming your deadline and each member's available hours. Do not start with all dashboards, all database tables or all backend code in isolation; build a small screen-to-database feature first.

Every feature follows the same development loop

Read its story -> sketch screen -> agree request/response -> create migration -> implement repository/service/controller -> connect React -> test happy and forbidden paths -> peer review -> merge -> update documentation. Ask a teammate to run the demo, not just read the code.

Git and review

One protected main branch; short feature branches; small pull requests. Pull main frequently and integrate at each checkpoint. Reviewer pairs: M1/M6, M2/M9, M3/M4, M5/M7; M8 reviews event integration with M6 and receives review from M9. Rotate shared setup work so leadership does not absorb everyone's integration burden.

Using a coding assistant responsibly

Give it the assignment, this design and the repository. Request one story with acceptance tests at a time. Require it to inspect existing code, explain the changed classes and report actual checks. You must be able to explain the request flow, business rules and database changes during the demo. Do not submit generated test results or claim unrun features work.

Quality, risk and operational decisions

Risk	Control	Verification
Task too large for beginners	Required scope first; extension backlog separate; checkpoints rather than big-bang integration.	Leader reviews progress and redistributes work weekly.
Unfair workload	Estimate stories together; reserve review/integration time; use screen/rule/table/test evidence.	Re-estimate after checkpoint C; same story count is not equal effort.
Authentication bottleneck	Publish fixed test principals and IdentityContext early; replace mocks in integrated tests.	M1 and one business feature integrate before all auth screens finish.
Broken migrations	Single ordered history and peer review; complete DB schema per machine.	Fresh PostgreSQL migration plus upgrade test in CI.
Lost or duplicate events	Transactional outbox, durable queues, acknowledgements and deduplication.	Broker-down/restart and repeated-event tests.
Data exposure	Role plus ownership checks; safe public tracking; private proof and redacted logs.	Negative authorization tests for every feature.
Concurrency conflicts	DB constraints, shared lock order, expectedVersion.	Two simultaneous assignment/status requests.
External-service outage	Timeouts, bounded retries, visible job failures and mock demo provider.	Simulated Google/SMS/webhook failures; no false success.
Deployment failure	Document env values, health checks, volumes and restore procedure.	Fresh machine/startup rehearsal and backup restore.

Operational baseline

Record request correlation IDs, safe errors, queue backlog, failed delivery jobs and DB migration version. Back up PostgreSQL and any private proof storage, then prove a restore before claiming recoverability. Configure retention periods with the team/lecturer rather than inventing a legal requirement. Use health endpoints that do not expose secrets.

Non-functional acceptance

Apply the measurable student-demo load targets on page 16; record hardware, dataset, duration, percentiles and failures. Run keyboard/mobile tests on real core journeys. Automated tests cover business rules and forbidden actions. The assignment requires measurable coverage but no specified percentage; report the actual result and important gaps.

Decisions still to record

Team names, group-size approval, submission date, hosting access, Google project owner, real or synthetic rates, supported zones, proof policy, privacy retention, secrets management and deployment restore owner. Each decision needs an owner, date, rationale and effect on tests. Requirements changes update this document and contracts together.

Documentation plan, glossary and references

Deliverable	Contents	Accountability
Design PDF (this document)	Scope, layers, use cases, ER, sequences, security, integration, feature plans and tests.	Leader coordinates; each member reviews their feature.
README	Prerequisites, pinned versions, env setup, startup, tests and troubleshooting.	M5 coordinates; all verify locally.
OpenAPI + event schemas	Endpoints, DTO examples, roles, errors, payload versions and idempotency.	M2 assembles; each feature owner authors.
Database dictionary	Table/column names, types, nullability, PK/FK, indexes, owner and retention.	M3 coordinates migration review; all own their tables.
Test report / coverage	Actual command output, scenarios, failures, fixes and coverage scope.	M9 assembles; all supply evidence.
Contribution sheet	Member name, story IDs, commits, screens, services, tables, tests and demo.	Leader assembles; members verify accuracy.
Demo runbook	Google login, quote/book/track, dispatch/courier, merchant order/webhook, notification.	M4 coordinates with all owners.
Decision/change log	Approved changes and unresolved items; reconcile code, diagrams and contracts.	Leader plus affected owners.

Glossary

API: an agreed way for software to request an operation. JWT: a signed token carrying identity/session claims. OAuth/OIDC: delegated authorization and identity protocols used for Google login. SSE: a long-lived server-to-browser event stream. Broker: software that holds/routes messages. Webhook: an HTTP callback to a partner. Idempotency: retrying an operation does not repeat its business effect. Projection: a stored view derived from source events. Migration: a versioned database change.

Technical references checked 31 October 2026

[T1] Spring Framework, Declarative Transaction Management: https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative.html

[T2] Spring Data JPA, repository overview: https://docs.spring.io/spring-data/jpa/reference/jpa.html

[T3] Spring Security, Servlet Architecture: https://docs.spring.io/spring-security/reference/servlet/architecture.html

Use documentation matching the team's pinned Spring Boot dependency versions; current reference pages may describe a newer major release. Assignment and prior design provenance are listed on page 18. This revision supersedes prior nine-schema/independent-runner assumptions with one Java layered backend and one shared application schema. It does not remove the required broker integration or JWT security.

Handover rule: this is a design and implementation plan, not a finished-project report. Before submission, update diagrams to match the actual code, fill member names and decisions, attach real test evidence and remove claims for any feature that was not delivered.

Design-documentation marking checklist

This section maps every item in the supplied 20-mark design-documentation checklist to concrete diagrams and explanations. The architecture is Java layered architecture deployed as one backend. The marking total is supplied by the assignment; no sub-mark allocation is assumed.

Required item	Where to find it	What is provided
Use-case diagrams: customer, merchant, courier and dispatcher	Pages 5-6	Four named human actors, system boundaries, use cases and associations; supporting preconditions and outcomes.
ER diagram: users, roles, merchants, parcels, status history, couriers, zones and pricing	Pages 34-36; overview pages 7-8	Linked ER views with separate entities, primary/foreign keys, cardinalities, ownership and integrity constraints.
Architecture proposal with justification and clear layers	Pages 3-4, 27-28 and 33	Java controllers, services, repositories and PostgreSQL; React UI, security, external integrations and deployment rationale.
Integration patterns and rationale	Page 15 and page 38	REST request/reply, RabbitMQ publish/subscribe, durable queues, outbox, retries, deduplication and SSE.
Creating a delivery sequence	Page 11	Quote, validation, idempotent booking and atomic persistence; alternate/failure paths.
Merchant API order sequence	Page 12	Credential/approval checks, server-derived business scope, shared booking service and duplicate handling.
Courier status update triggering notifications	Pages 13 and 38	Committed state/outbox followed by broker fan-out and explicit notification-provider call.
OAuth 2.0 login sequence	Page 14	Google authorization code, state/nonce/PKCE, verified identity and application JWT/session establishment.
Design patterns and principles	Page 37	SOLID, DRY, Repository, Strategy and Observer-style publish/subscribe with system-specific examples.

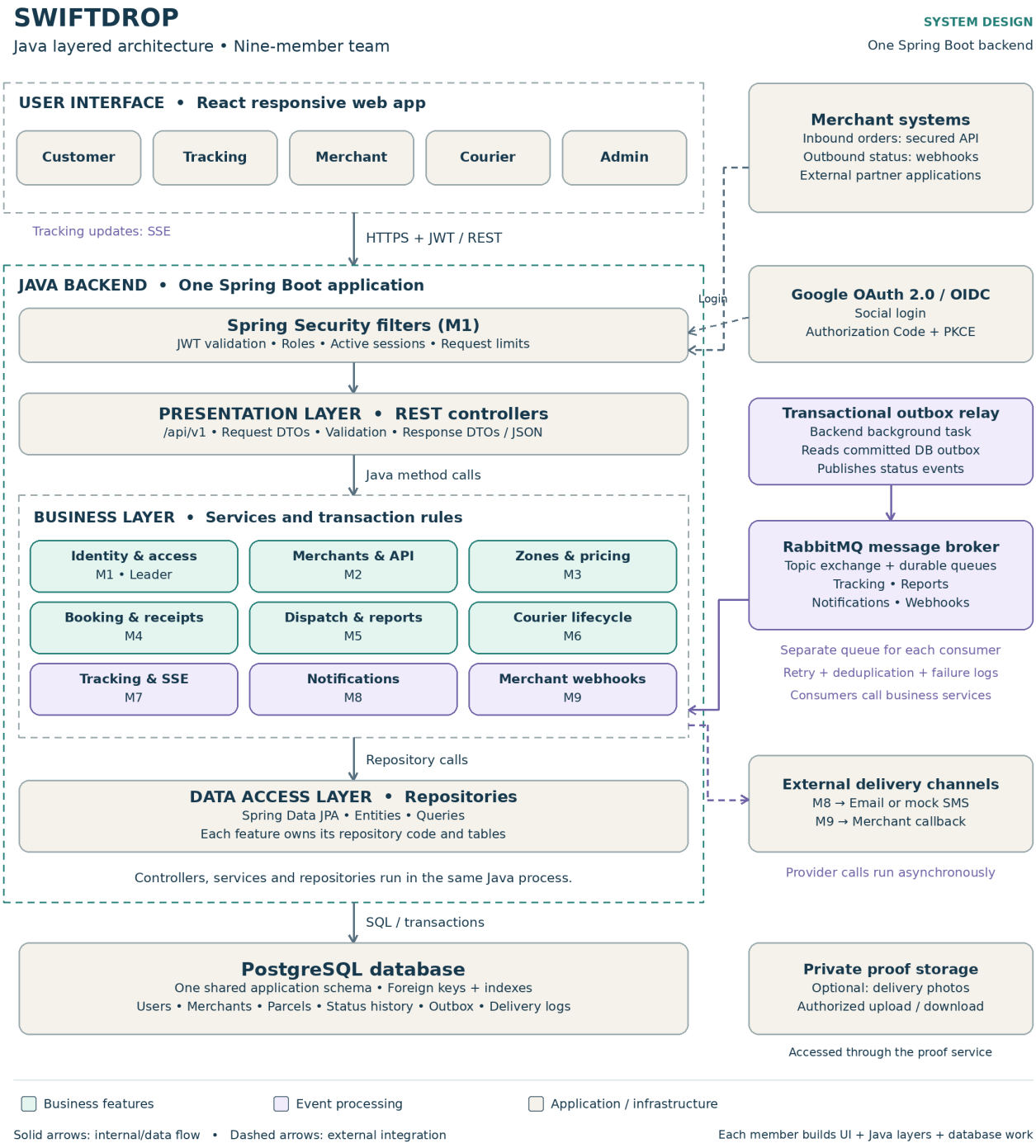
How the diagrams fit together

Use cases describe who uses the system and for what goal. ER diagrams describe persistent data relationships. Architecture diagrams describe responsibilities and dependencies. Sequence diagrams describe the order of interactions for a particular operation. None of these replaces the others.

Review before submission

Use this document as the design baseline. Fill team names and student IDs in your contribution sheet; verify the lecturer accepts nine members; update diagrams if the implementation changes. Attach actual test and coverage results separately. The document does not claim that the proposed code or integrations have already been implemented.

Complete system architecture diagram

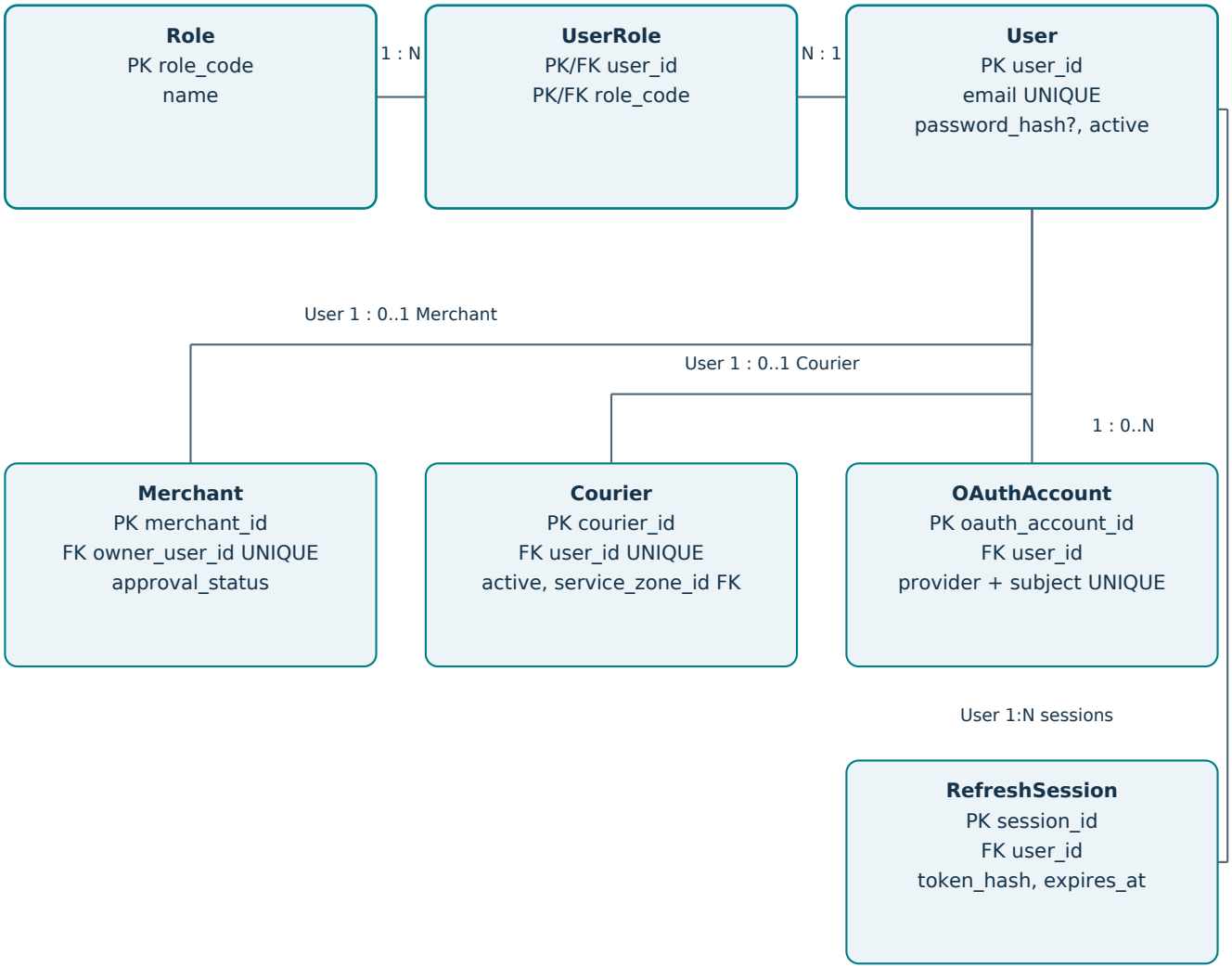


Architecture justification

Layer separation keeps HTTP handling, business decisions and persistence independently understandable and testable. One Java backend reduces network/deployment complexity for a beginner team, while service interfaces preserve feature ownership. PostgreSQL transactions enforce atomic booking and lifecycle changes. RabbitMQ is used for asynchronous side effects, not as a replacement for every internal Java call.

The side panel groups integrations for readability. The outbox relay and consumers are background components inside the initial backend process; RabbitMQ and external providers are separate systems. The controller, service and repository layers are not separate deployed services. M2 additionally authenticates inbound merchant API credentials.

ER diagram A - identity and business actors

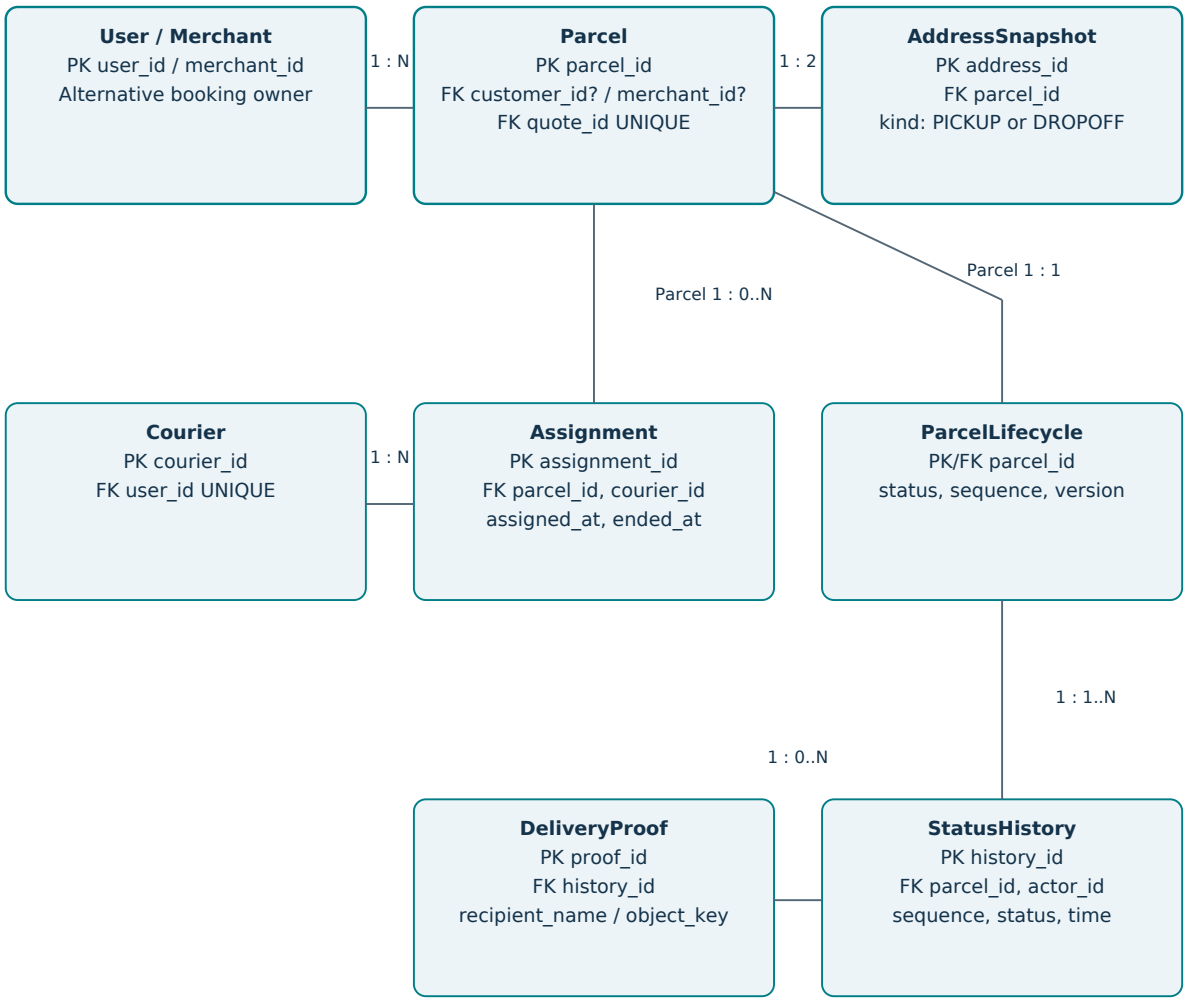


Notation: PK = primary key; FK = foreign key; 1 = exactly one; 0..1 = optional single record; N = many. Lines show relationships, not request flow. Every foreign-key child references exactly one parent unless explicitly nullable. UserRole resolves the many-to-many User/Role relationship.

Relationship	Constraint / meaning
User to Merchant	One owner may have at most one business in this baseline; a merchant has one owner. Multi-staff organizations are an extension.
User to Courier	Operational courier profile is optional and unique per user; role alone does not create the profile.
User to OAuthAccount	Multiple provider identities can link explicitly to a user; provider/subject is globally unique.
User to RefreshSession	A user can have multiple sessions; tokens are stored as hashes and revocable independently.

Links to other ER views: User is referenced by Parcel.customer_id and StatusHistory.actor_id. Merchant is referenced by Parcel.merchant_id. Courier participates in Assignment and optionally references its service Zone.

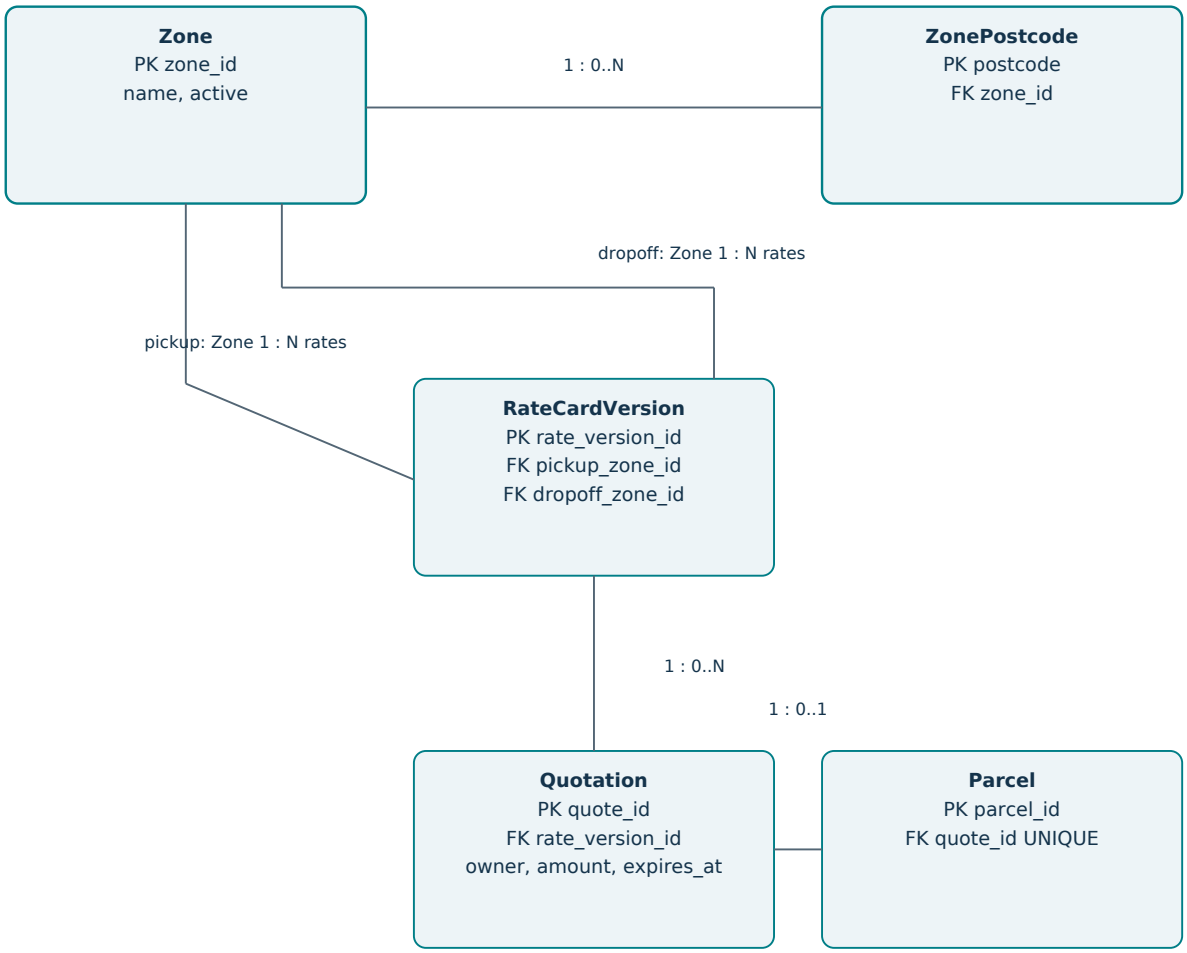
ER diagram B - parcels, assignments and history



Invariant	Database / service enforcement
Exclusive owner	Exactly one of customer_id and merchant_id is non-null; both are foreign keys to their own parent tables.
Addresses	UNIQUE(parcel_id, kind) and restricted kinds; booking transaction creates both required snapshots.
Assignments	Many historical assignments, at most one current row per parcel using a partial unique index.
History	UNIQUE(parcel_id, sequence); initial CREATED history is committed with the parcel/lifecycle.
Proof	History row can have zero or more evidence records; successful delivery requires the configured evidence.

User/ Merchant at the top denotes two alternative owner relationships, not one physical table. An actor FK on StatusHistory points to User (ER A). The FK from StatusHistory.parcel_id references ParcelLifecycle.parcel_id. Tracking projections are derived from these records and cannot change their canonical status.

ER diagram C - zones, rate versions and quotes



Rate data and versioning

Each RateCardVersion stores currency, base_minor, included_kg, extra_kg_minor, size_surcharge_S/M/L, effective_from and active state. Each record references exactly one pickup zone and one drop-off zone. Prevent overlapping active rate versions for the same route/service period in the pricing service; use a database exclusion constraint if intervals are modeled directly.

Quotation data

Quotation also stores customer_id or merchant_id as nullable FKs with an exclusive-owner check, input_hash, weight_grams, size, amount_minor and the rate/price breakdown snapshot. The immutable version reference explains how the quote was calculated; the saved snapshot protects historical meaning. A quote may be unconsumed or referenced by one parcel.

Data design interpretation

RateCardVersion is the concrete versioned rate_cards record described in the feature plan. It is separate from Quotation: rates are configuration, quotes are customer-specific results. The combined RateCard / Quote box on page 7 is only a compact overview; this ER view is the detailed implementation model. ZonePostcode uses one configured zone per postcode in this baseline.

Together, ER views A-C cover every required domain: users, roles, merchants, parcels, status history, couriers, zones and pricing. API credentials, notification jobs, webhook deliveries and their retry attempts are supporting tables specified in the feature plans and page 8.

Design patterns and principles applied

Principle / pattern	Concrete SwiftDrop application	Why it is useful
Single responsibility (S)	ParcelController handles HTTP; BookingService enforces booking rules; ParcelRepository persists records.	A validation/UI change does not require rewriting database access.
Open/closed (O)	PricingStrategy allows a new fee policy through a new implementation selected by configuration.	Extend fee behaviour while keeping booking orchestration stable.
Liskov substitution (L)	A mock SMS adapter and a real provider implement the same success/failure and retry contract.	Tests and production can substitute implementations without changing caller expectations.
Interface segregation (I)	SenderContactQuery exposes only the contact lookup needed by notifications; it does not expose all booking mutations.	Small interfaces reduce coupling and unnecessary access.
Dependency inversion (D)	NotificationService depends on an SmsGateway interface; constructor injection supplies the adapter.	Business rules do not depend directly on a particular vendor HTTP client.
DRY	Customer and merchant booking call the same BookingService and PricingService.	One rule implementation avoids inconsistent quotes and duplicate validation logic.
Repository	Feature-owned Spring Data repositories encapsulate persistence operations and queries.	Services can focus on rules and be tested with controlled dependencies.
Strategy	Fee = route base + excess rounded weight charge + configured size surcharge. Alternative policies implement the same contract.	Pricing choices are explicit and testable; avoid embedding formulas in controllers.
Observer / publish-subscribe	One committed ParcelStatusChanged event is observed by tracking, reports, notification and webhook consumers.	New subscribers can react without adding provider calls to the courier request.
Transactional outbox	Business changes and outgoing event record commit in one DB transaction; relay publishes later.	A broker outage cannot silently discard a successfully committed status event.
Idempotent consumer	Deduplicate by event/target; preserve original eventId on retry and replay.	Repeated delivery does not repeat logical jobs or regress tracking state.

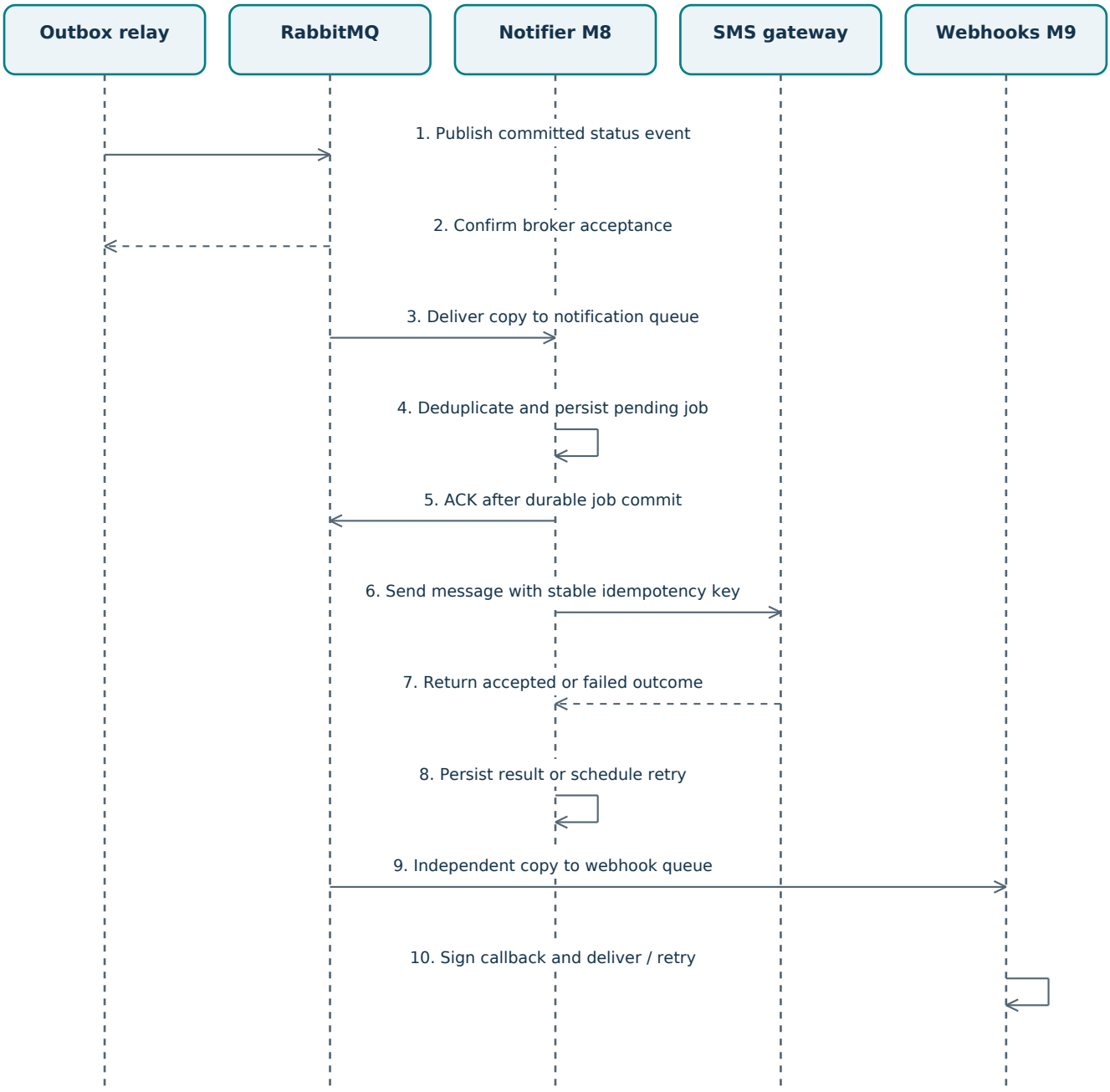
Observer versus distributed messaging

A traditional Observer calls listeners inside one process. RabbitMQ publish/subscribe applies the same notification idea across asynchronous consumers, with additional durability, acknowledgement, retry and duplicate-handling needs. Do not describe the broker as exactly-once delivery or rely on an in-memory event alone for durable integration.

Apply patterns only where useful

Interfaces and patterns support clear rules and testability; they are not marks earned by adding empty classes. Each teammate should demonstrate one real example from their implementation. This section describes planned application of the patterns, not a claim that the code already exists.

Sequence detail - status event to notification



Precondition: page 13 has already committed the courier status, history and outbox in one transaction. This diagram expands the subsequent asynchronous delivery; it does not delay or roll back the courier HTTP response. Tracking and reports receive their own copies through separate queues, omitted here to keep the notification path readable.

Delivery and failure semantics

The notifier stores a durable job before acknowledging the broker message. A worker sends that job and records provider outcome; pending jobs recover after a restart. Provider acceptance is not proof the user read or received the message. On failure, persist the next attempt time; after exhaustion expose a controlled retry action. The gateway may be mock SMS as permitted by the assignment.

The relay marks its event published only after broker confirmation. If confirmation is lost, republishing may duplicate the event; deduplication protects logical jobs. The webhook consumer independently resolves merchant subscriptions and signs callbacks. Its failures do not prevent notification or tracking processing.